Core Java Interview Questions

17 essential questions with explanations and code examples

1. What Is the Difference Between JDK, JRE, and JVM?

Imagine JVM as a virtual engine that runs Java programs, JRE as a ready-to-use car with the engine installed, and JDK as a complete workshop with tools, parts, and manuals for building and maintaining cars.

JVM (Java Virtual Machine)

Definition: JVM is an abstract computing machine that enables Java bytecode to be executed on different hardware platforms.

Functionality: It interprets Java bytecode into machine code or executes it using Just-In-Time (JIT) compilation, providing platform independence for Java applications.

Use Case: Suppose you have a Java application compiled into bytecode. The JVM executes this bytecode, translating it into machine-specific instructions for the underlying hardware.

JRE (Java Runtime Environment)

Definition: JRE is a runtime environment that contains JVM, libraries, and other components required to run Java applications but does not include development tools.

Functionality: It provides the runtime environment necessary for executing Java applications, including the JVM, core libraries, and additional runtime resources.

Use Case: When you want to run a pre-built Java application on your system, you need JRE installed. It provides the necessary runtime environment for the application to execute.

JDK (Java Development Kit)

Definition: JDK is a software development kit that includes JRE, development tools, libraries, and documentation necessary for developing Java applications.

Functionality: It provides everything included in JRE, along with additional tools such as compilers, debuggers, and development APIs for creating Java applications.

Use Case: If you're a Java developer, you'll use JDK to write, compile, and debug Java code. It includes everything you need for Java development, from writing code to deploying applications.

2. What Is the Difference Between Stack and Heap?

The stack and heap are two types of memory storage in the Java Virtual Machine (JVM).

The stack is used for storing method invocation and local variables. Each thread in a program has its own stack, and it operates in a last-in, first-out manner. When a method is called, a new stack frame is created, and when the method execution is completed, the stack frame is removed. Stack memory is typically faster to access than heap memory.

The heap is a shared memory pool used for dynamic memory allocation. Objects created in Java are stored in the heap. Unlike the stack, memory in the heap can be accessed randomly. Garbage collection is responsible for managing memory in the heap by reclaiming memory occupied by objects that are no longer needed.

In summary, the stack is used for storing method call information and local variables with a predictable, limited size per thread, while the heap is used for dynamic memory allocation of objects with a larger, shared memory space.

Always remember that the stack has much less memory space than the heap and when a stack is full, the JVM will throw a StackOverflowError.

3. Why is the main method in Java declared as public static void main(String[] args)?

The main method is declared this way for the following reasons:

- **public:** So the JVM can access it from outside the class.
- **static:** Allows the JVM to invoke it without creating an instance of the class.
- **void:** Indicates that the method does not return any value.
- **String[] args:** Used to accept command-line arguments when the program runs.

4. What Is the void Type and When is it preferable to Use It?

In Java, the void type is used to indicate that a method does not return any value. Using void appropriately makes the code more readable and helps communicate the intent of the method to other developers. It indicates that the method performs an action or has a side effect without returning any result.

```
// Example 1: Method with side effects
void writeToLogFile(String message) {
    // Write message to log file
}

// Example 2: Event handler method
void onButtonClick() {
    // Perform action when button is clicked
}

// Example 3: Setter method
void setName(String name) {
    this.name = name;
}

// Example 4: Utility method
void printMessage(String message) {
    System.out.println(message);
}

// Example 5: Lifecycle method
void init() {
    // Initialize resources
}
```

5. Which Access Modifiers Are Available in Java and their Purpose?

In Java, there are four access modifiers available, each serving a specific purpose:

Public (public)

Purpose: Allows unrestricted access to the member (class, method, or field) from any other class or package.

Example: A public method in a class can be accessed by any other class, even if it's in a different package.

Private (private)

Purpose: Restricts access to the member to only within the same class.

Example: A private field in a class can only be accessed and modified by methods within the same class.

Protected (protected)

Purpose: Allows access to the member within the same package and by subclasses (whether they are in the same package or different package).

Example: A protected method in a superclass can be accessed by its subclass, regardless of whether the subclass is in the same package or a different package.

Default (Package-Private)

Purpose: Restricts access to the member to only within the same package. If no access modifier is specified, it is considered default (package-private).

Example: A default (package-private) class member can be accessed by any class within the same package but not by classes outside the package.

These access modifiers help control the visibility and accessibility of classes, methods, and fields, allowing developers to enforce encapsulation, data hiding, and code organization in their Java programs.

6. Why Is String an Immutable Class?

String is immutable in Java, meaning its state cannot be changed after it's created. There are several reasons why String is designed to be immutable:

Thread Safety

Immutable objects are inherently thread-safe because their state cannot be modified once created. This eliminates the need for synchronization when working with Strings in a multi-threaded environment.

String Pool Optimization

Strings are stored in a string pool, which is a pool of unique String literals in memory. Immutable strings allow for string interning, where multiple references to the same string literal point to the same memory location. This reduces memory overhead and improves performance.

Security

Immutable strings are inherently more secure because their values cannot be changed once created. This prevents malicious code from modifying sensitive string data.

Caching and Optimization

Immutable strings can be cached and reused, leading to performance optimizations in applications. Operations like substring, concatenation, and conversion to upper/lower case can be optimized by reusing existing string instances.

Hashing and Hashcode Stability

Immutable strings have a stable hashcode, meaning their hashcode value remains constant throughout their lifetime. This property is essential for objects used in hash-based collections like HashMap and HashSet.

7. Explain Immutable nature of String with example?

```
public static void main(String[] args) {
    // Creating a string "Hello" and assigning it to str1 (stored in the string pool in the
    heap)
    String str1 = "Hello";

    // Creating another string "Hello" (refers to the same object in the string pool)
```

```
    String str3 = "Hello"; // Due to string interning, it refers to the same object in the
    string pool

    // Concatenating "World" to str1 and assigning it to str2
    String str2 = str1 + " World"; // New string created and stored in the heap

    // Converting str1 to uppercase and assigning it to str1
    str1 = str1.toUpperCase(); // New string created and stored in the heap

    // Outputting the values of str1, str2, and str3
    System.out.println("str1: " + str1); // Output: "HELLO"
    System.out.println("str2: " + str2); // Output: "Hello World"
    System.out.println("str3: " + str3); // Output: "Hello"
}
```

8. What is the concept of String Intern?

The concept of string internment in Java refers to the process of storing unique string literals in a special memory area known as the "string pool." When a string literal is encountered in Java code, the JVM checks if an equivalent string already exists in the string pool. If it does, the JVM returns a reference to the existing string object from the pool rather than creating a new object.

This concept ensures that only one copy of each distinct string literal is maintained in memory, reducing memory usage and improving performance by facilitating string sharing and reuse.

String interning can be explicitly invoked using the intern() method provided by the String class. When called on a string object, the intern() method checks if an equivalent string already exists in the string pool. If not, it adds the string to the pool and returns a reference to it. If an equivalent string already exists, it returns a reference to the existing string object from the pool.

```
public static void main(String[] args) {
    String str1 = "Hello";
    String str2 = "Hello";
    String str3 = new String("Hello").intern();

    System.out.println(str1 == str2); // Output: true (Same object reference)
    System.out.println(str1 == str3); // Output: true (Same object reference)
}
```

9. What happens when you create a String with a new keyword?

When the new keyword is used to create a string object, the JVM creates the object in the heap memory, regardless of whether an equivalent string already exists in the string pool. If the string is not found in the string pool, the JVM creates a new string object in the heap and returns a reference to it.

Here's a simplified explanation of the process:

When new keyword is used to create a string object, the JVM creates the object in the heap memory. If the string already exists in the string pool, a new object is still created in the heap, and the string is not added to the pool again.

If the string does not exist in the string pool, the JVM adds it to the pool and returns a reference to the newly created object in the heap.

This behavior ensures that each call to new String("...") creates a new string object in the heap, regardless of whether an equivalent string exists in the string pool.

This can be useful in scenarios where you explicitly want to avoid string interning and ensure that a new object is created in the heap. However, it's important to note that using new unnecessarily can lead to

unnecessary memory usage and should be done judiciously.

```
public static void main(String[] args) {
    // Creating a string object using the new keyword
    // This creates a new object in the heap memory
    String str1 = new String("Hello");

    // Creating another string object with the same value
    // This also creates a new object in the heap memory
    String str2 = new String("Hello");

    // Checking if str1 and str2 reference the same object
    // They are not the same object because each call to new creates a new object
    System.out.println("str1 == str2: " + (str1 == str2)); // Output: false

    // Creating a string object without using the new keyword
    // This checks the string pool and creates a new object only if the string is not already
    present
    String str3 = "Hello";

    // Checking if str1 and str3 reference the same object
    // They are not the same object because str3 is created in the string pool
    System.out.println("str1 == str3: " + (str1 == str3)); // Output: false
}
```

10. What is JAR vs WAR vs EAR?

JAR, WAR, and EAR are all archive file formats used in Java for packaging and deploying applications. Here's a brief overview of each:

JAR (Java ARchive)

Purpose: JAR files are used to package Java class files, resources, and metadata into a single archive file.

Contents: They typically contain compiled Java classes, libraries (JAR files), configuration files, and resources (images, properties files, etc.).

Usage: JAR files are commonly used for packaging libraries, standalone applications, and executable Java applications.

Extension: .jar

WAR (Web ARchive)

Purpose: WAR files are specifically used for packaging and deploying web applications in Java EE (Enterprise Edition) environments.

Contents: They include web components such as servlets, JSP files, HTML pages, JavaScript files, CSS stylesheets, and other resources needed for web applications.

Structure: WAR files have a defined directory structure (e.g., WEB-INF) to organize web application resources and configuration files.

Usage: WAR files are deployed to servlet containers (e.g., Apache Tomcat, Jetty) to run Java web applications.

Extension: .war

EAR (Enterprise ARchive)

Purpose: EAR files are used for packaging and deploying enterprise-level Java EE applications, which typically consist of multiple modules and components.

Contents: They can contain one or more WAR files, EJB (Enterprise JavaBeans) JAR files, application client JAR files, library JAR files, and other resources.

Structure: EAR files have a hierarchical structure, with modules organized into directories and subdirectories.

Usage: EAR files are deployed to Java EE application servers (e.g., JBoss, WebSphere, WebLogic) to run large-scale enterprise applications.

Extension: .ear

In summary, JAR files are generic archive files used for packaging Java applications and libraries. WAR files are specialized for web applications and contain web-specific resources. EAR files are used for large-scale enterprise applications containing multiple modules and components. Each type of archive serves a specific purpose in Java application development and deployment.

11. Java is strictly pass-by-value language?

Yes, Java is strictly pass-by-value language. When you pass arguments to a method in Java, the values of the arguments are copied and passed to the method parameters.

However, it's important to note that the behavior can sometimes be confusing, especially when dealing with objects. When you pass an object to a method, you are passing a reference to the object, not the object itself. This reference is also passed by value, meaning a copy of the reference is passed to the method. This can lead to the misconception that Java uses pass-by-reference, but in reality, it's still pass-by-value.

Here's a brief summary:

- For primitive data types (int, double, boolean, etc.), the value itself is passed.
- For objects, the reference to the object is passed by value. This means changes to the object's state made inside the method are reflected outside the method, but reassigning the reference inside the method does not affect the reference outside the method.

```
public class PassByValueExample {
    public static void main(String[] args) {
        int num = 10;
        System.out.println("Before calling modifyValue: num = " + num);

        // Passing a primitive type (int) by value
        modifyValue(num);

        System.out.println("After calling modifyValue: num = " + num);

        StringBuilder sb = new StringBuilder("Hello");
        System.out.println("Before calling modifyReference: sb = " + sb);

        // Passing an object (StringBuilder) by value (passing reference by value)
        modifyReference(sb);

        System.out.println("After calling modifyReference: sb = " + sb);
    }

    // Method to modify a primitive value
    public static void modifyValue(int num) {
        num += 5;
        System.out.println("Inside modifyValue: num = " + num);
    }

    // Method to modify an object reference
    public static void modifyReference(StringBuilder sb) {
        sb.append(" World");
        System.out.println("Inside modifyReference: sb = " + sb);
```

```
    }
}

// Output
// Before calling modifyValue: num = 10
// Inside modifyValue: num = 15
// After calling modifyValue: num = 10
// Before calling modifyReference: sb = Hello
// Inside modifyReference: sb = Hello World
// After calling modifyReference: sb = Hello World
```

12. Explain about static, abstract and final keywords in Java?

static

- The static keyword in Java is used to declare members (variables and methods) that belong to the class itself, rather than to instances of the class.
- Variables declared as static are called class variables or static variables. They are shared among all instances of the class and can be accessed using the class name.
- Methods declared as static are called class methods or static methods. They belong to the class itself and can be called without creating an instance of the class.
- static variables and methods can be accessed directly using the class name, without the need to create an object of the class.

abstract

- The abstract keyword in Java is used to declare abstract classes and methods.
- Abstract classes cannot be instantiated directly; they serve as templates for concrete subclasses to extend and implement.
- Abstract methods are declared without an implementation and must be implemented by concrete subclasses.
- Abstract classes may contain both abstract and concrete methods, but a class with at least one abstract method must be declared abstract.
- Abstract methods are intended to be overridden by subclasses to provide specific implementations.

final

- The final keyword in Java is used to restrict the modification of classes, methods, and variables.
- When applied to a class, the final keyword prevents the class from being subclassed (i.e., it becomes a final class).
- When applied to a method, the final keyword prevents the method from being overridden by subclasses.
- When applied to a variable, the final keyword indicates that the variable's value cannot be changed after initialization. For primitive types, it makes the value immutable, while for reference types, it makes the reference immutable (i.e., the reference cannot be changed, but the object's state can).
- final variables must be initialized either when declared or in a constructor, and they cannot be reassigned.

In summary, the static keyword is used to define members that belong to the class itself, the abstract keyword is used to declare abstract classes and methods, and the final keyword is used to restrict modification of classes, methods, and variables.

```
// StaticExample class with a static variable and method
class StaticExample {
    // Static variable
```

```
    static int staticVariable = 10;

    // Static method
    static void staticMethod() {
        System.out.println("Inside staticMethod: staticVariable = " + staticVariable);
    }
}

// AbstractExample abstract class with an abstract method
abstract class AbstractExample {
    // Abstract method (no implementation)
    abstract void abstractMethod();
}

// Concrete subclass of AbstractExample
class ConcreteExample extends AbstractExample {
    // Implementation of abstract method
    void abstractMethod() {
        System.out.println("Inside abstractMethod in ConcreteExample");
    }
}

// FinalExample class with final variable and method
class FinalExample {
    // Final variable
    final int finalVariable = 20;

    // Final method
    final void finalMethod() {
        System.out.println("Inside finalMethod: finalVariable = " + finalVariable);
    }
}

public class KeywordExample {
    public static void main(String[] args) {
        // Accessing static variable and method using class name
        System.out.println("Static variable: " + StaticExample.staticVariable);
        StaticExample.staticMethod();

        // Creating an instance of ConcreteExample and calling abstractMethod
        ConcreteExample concrete = new ConcreteExample();
        concrete.abstractMethod();

        // Creating an instance of FinalExample and accessing final variable and method
        FinalExample finalExample = new FinalExample();
        System.out.println("Final variable: " + finalExample.finalVariable);
        finalExample.finalMethod();
    }
}

/* Output
Static variable: 10
Inside staticMethod: staticVariable = 10
Inside abstractMethod in ConcreteExample
Final variable: 20
Inside finalMethod: finalVariable = 20
*/
```

13. Difference between String, StringBuffer, and StringBuilder?

The main difference between String, StringBuffer, and StringBuilder in Java lies in their mutability, thread safety, and performance characteristics:

String

- String objects are immutable, meaning their values cannot be changed after they are created.

- Once a String object is created, its value cannot be modified. Any operation that appears to modify a String actually creates a new String object with the modified value.
- String objects are thread-safe because they are immutable, making them safe for use in multi-threaded environments.
- Due to their immutability, String objects are safe for sharing across different threads without synchronization overhead.
- Use String when you need an immutable sequence of characters.

StringBuffer

- StringBuffer is a mutable sequence of characters, designed for use in multi-threaded environments.
- StringBuffer provides synchronized methods for appending, inserting, and modifying characters, making it thread-safe.
- Modifications to StringBuffer objects are performed in-place, avoiding unnecessary memory allocation and string copying.
- Use StringBuffer when you need a mutable sequence of characters in a multi-threaded environment.

StringBuilder

- StringBuilder is similar to StringBuffer in that it provides a mutable sequence of characters. However, it is not inherently thread-safe.
- Unlike StringBuffer, StringBuilder does not provide synchronized methods, making it more efficient in single-threaded scenarios.
- StringBuilder is recommended for use in single-threaded environments or when synchronization is not required.
- Use StringBuilder when you need a mutable sequence of characters in a single-threaded environment.

In summary, String is immutable and thread-safe, StringBuffer is mutable and thread-safe, and StringBuilder is mutable but not inherently thread-safe. The choice between them depends on the specific requirements of your application, such as mutability, thread safety, and performance considerations.

```
public static void main(String[] args) {
    // String Example
    String str = "Hello";
    System.out.println("Original String: " + str);

    // Concatenating a new string to the original string
    str = str + " World";
    System.out.println("Modified String (using String): " + str);

    // StringBuffer Example
    StringBuffer stringBuffer = new StringBuffer("Hello");
    System.out.println("Original StringBuffer: " + stringBuffer);

    // Appending a new string to the original StringBuffer
    stringBuffer.append(" World");
    System.out.println("Modified StringBuffer: " + stringBuffer);

    // StringBuilder Example
    StringBuilder stringBuilder = new StringBuilder("Hello");
    System.out.println("Original StringBuilder: " + stringBuilder);

    // Appending a new string to the original StringBuilder
    stringBuilder.append(" World");
    System.out.println("Modified StringBuilder: " + stringBuilder);
}
```

14. How Can We Reverse a String in Java?

The common approach to reverse a string in Java is by using either StringBuilder or StringBuffer along with their reverse() method as shown below:

```
String reversed_sBuilder = new StringBuilder("hungryCoders").reverse().toString();
System.out.println(reversed_sBuilder); // sredoCyrgnuh

String reversed_sBuffer = new StringBuffer("hungryCoders").reverse().toString();
System.out.println(reversed_sBuffer); // sredoCyrgnuh
```

15. What Is String.format() and How to Use It?

String.format() is a method in Java that allows you to create formatted strings by substituting placeholders with values.

```
String formattedString = String.format("Hello, %s! Welcome to %s.", "Friend", "HungryCoders.
    com");
System.out.println(formattedString); // Hello, Friend! Welcome to HungryCoders.com.
```

16. What Is an Enum and How to Use It?

An enum, short for "enumeration," is a special data type in Java used to define a collection of constants. It provides a way to represent a fixed set of predefined values.

```
// Defining an enum called DayOfWeek
enum DayOfWeek {
    SUNDAY, MONDAY, TUESDAY, WEDNESDAY, THURSDAY, FRIDAY, SATURDAY
}

public class EnumExample {
    public static void main(String[] args) {
        // Using enum constants
        DayOfWeek today = DayOfWeek.MONDAY;
        System.out.println("Today is: " + today);

        // Switch statement with enum
        switch (today) {
            case MONDAY:
                System.out.println("It's the start of the week.");
                break;
            case FRIDAY:
                System.out.println("TGIF!");
                break;
            default:
                System.out.println("Just another day.");
        }

        // Iterating over enum constants
        System.out.println("Days of the week:");
        for (DayOfWeek day : DayOfWeek.values()) {
            System.out.println(day);
        }
    }
}

/* Output
Today is: MONDAY
It's the start of the week.
Days of the week:
```

```
SUNDAY
MONDAY
TUESDAY
WEDNESDAY
THURSDAY
FRIDAY
SATURDAY
*/
```

17. What Are Two Types of Casting in Java?

In Java, implicit casting (widening) is a form of upcasting, while explicit casting (narrowing) is a form of downcasting.

Upcasting (Implicit Casting)

- Upcasting involves converting an object of a subclass type to an object of its superclass type.
- It is performed automatically by the compiler when assigning a subclass object to a superclass reference.
- Upcasting is considered safe because it involves moving from a more specific type to a more general type.

Downcasting (Explicit Casting)

- Downcasting involves converting an object of a superclass type to an object of its subclass type.
- It requires an explicit cast operator and may result in a ClassCastException if the object being casted is not actually an instance of the target subclass.
- Downcasting is considered risky because it involves moving from a more general type to a more specific type, which may lead to runtime errors if not done properly.

In summary, implicit casting (upcasting) is converting from a subclass to a superclass type, while explicit casting (downcasting) is converting from a superclass to a subclass type. Upcasting is safe and implicit, while downcasting requires an explicit cast operator and may result in runtime errors if not done properly.

```
int num1 = 10;
double num2 = num1; // Implicit casting from int to double
System.out.println(num2); // 10.0

double num3 = 10.5;
int num4 = (int) num3; // Explicit casting from double to int
System.out.println(num4); // 10
```

18. Is Java statically and strongly typed? Explain in detail?

Java is both Statically and Strongly Typed.

Statically Typed: You cannot declare a variable without a type, and the type is checked at compile-time.

Strongly Typed: Type rules are strictly enforced, and incompatible types cannot mix without explicit conversions.

Statically Typed

- In Java, you must declare the type of every variable explicitly (or inferred, as with var introduced in Java 10)
- Type checking occurs during compilation. If there is a type mismatch, the compiler throws an error.
- Types are fixed after declaration and cannot change.

Being a statically typed language have lot of benefits such as:

- It prevents many runtime errors.
- It improves performance because type information is resolved during compilation.

```
int number = 10;  // Declared as int
number = "Hello"; // Compilation Error: Type mismatch
```

Strongly Typed

In Java, type conversions must follow strict rules, and incompatible types cannot be mixed without explicit conversion.

- Implicit type conversions (type coercion) are limited.
- Operations between incompatible types are not allowed.
- Java enforces strict type safety in collections (since Java 5 with generics)

Being a strongly typed language have lot of benefits such as:

- Avoids unpredictable behavior due to unintended type conversions.
- Ensures code reliability and readability.

```
int num = 10;
double result = num;  // Allowed: Implicit widening (int to double)
int value = result;   // Compilation Error: Explicit narrowing required

int x = 5;
String y = "10";
int sum = x + y; // Compilation Error: Incompatible types

List<String> list = new ArrayList<>();
list.add(42); // Compilation Error: Incompatible type
```

19. Does var keyword makes Java as a dynamically typed language?

Java introduced the var keyword in Java 10, which allows you to declare local variables without explicitly specifying their type. However, this does not mean Java has become a dynamically typed language. It is still statically and strongly typed, even with var.

Basically var is a reserved type name introduced to simplify local variable declarations. The compiler infers the type of the variable from the value assigned to it during initialization.

With var, the type is determined at compile-time, not at runtime. The compiler infers the type based on the assigned value and enforces type-checking.

This is still different from dynamically typed languages like Python or JavaScript, where variable types can change at runtime.

```
var number = 42;      // Compiler infers as int
number = "Hello";     // Compilation Error: Type mismatch
```

Even with var, Java still enforces strict rules about type usage and does not allow implicit type coercion.

```
var x = 10;         // Inferred as int
var y = "Hello";    // Inferred as String

int z = x + y;      // Compilation Error: Incompatible types
```

Key Restrictions of var:

- Cannot be used without initialization: The compiler needs the right-hand side value to infer the type.
- Type is fixed after inference: Once the type is inferred, it cannot change.
- Only for local variables: var cannot be used for Class fields, Method parameters and Return types
- Not allowed with null as an initializer

```
var num;  // Compilation Error: Cannot infer type for variable

var value = 10;  // Inferred as int
value = "Hello"; // Compilation Error: Type mismatch

var value = null; // Compilation Error: Cannot infer type for variable
```

20. What are the two main categories of data types in Java?

1. Primitive Data Types

These are the basic building blocks for data manipulation. They store simple values and include:

- boolean: true/false values.
- Numeric types like byte, short, int, long, float, and double.

- char: A single Unicode character.

2. Non-Primitive (Reference) Data Types

These store references (memory addresses) rather than the actual data. Examples:

- Strings: Represent sequences of characters.
- Arrays: Collections of elements of the same type.
- Classes and Objects: Represent complex structures with attributes and methods.

21. What are the 8 primitive data types in Java, and what does each represent?

Below are the 8 primitive data types in Java.

Boolean: Represents logical values (true or false).

```
boolean isJavaFun = true;
```

Byte: 8-bit signed integer. Used for Efficient memory storage, e.g., in large arrays.

```
byte b = 100;
```

Char: Represents a single 16-bit Unicode character. Used for Characters like a, 1, or symbols.

```
char grade = 'A';
```

Short: 16-bit signed integer. Used for Saving memory for small numbers.

```
short age = 200;
```

Int: 32-bit signed integer. Used for Default for whole numbers.

```
int salary = 50000;
```

Long: 64-bit signed integer. Used for Large numeric ranges.

```
long population = 8_000_000_000L;
```

Float: 32-bit single-precision floating-point. Used For decimals when precision is not critical.

```
float pi = 3.14f;
```

Double: 64-bit double-precision floating-point. Use for Default for decimals or scientific calculations.

```
double e = 2.718281828459045;
```

22. Why does char use 2 bytes in Java, and not 1 byte like in C/C++?

Java uses the Unicode system to support characters from all languages worldwide (e.g., Latin, Greek, Chinese).

But Unicode requires 2 bytes (16 bits) to represent all possible characters.

By contrast, ASCII, used in C/C++, is limited to 1 byte (8 bits), supporting only 256 characters.

23. What is the difference between float and double in Java? When would you use one over the other?

Double is more precise and better for computations requiring high accuracy, whereas float is more memory-efficient.

- Float is used when memory is limited (e.g., in embedded systems or large datasets).
- But Double is used as a default for decimal values, scientific and financial calculations.

```
float pi = 3.14159f; // Use 'f' to indicate float
double e = 2.718281828459045;
```

24. What is widening and narrowing type conversion? Provide examples of each.

Widening Conversion is process of Implicit conversion from a smaller to a larger type. Example: int to long or float to double

```
int num = 100;
double result = num; // Implicit widening
```

Narrowing Conversion is process of Explicit conversion from a larger to a smaller type (requires casting). Example: double to int

```
double value = 9.99;
int truncated = (int) value; // Explicit narrowing
```

25. What happens if you assign a larger data type to a smaller data type?

Java will throw a compile-time error, as narrowing requires explicit casting.

```
long bigNum = 100000L;
int smallerNum = bigNum; // Compilation Error
```

26. How does Java handle type safety in mixed-type arithmetic operations?

Java automatically promotes smaller types to the larger type in mixed operations.

```
int num = 5;
float value = 2.5f;
float result = num + value; // num promoted to float
```

27. When should you use byte and short instead of int?

Use byte and short when:

- Memory is constrained.
- You're working with fixed ranges (e.g., file streams or network protocols).

```
byte age = 127; // Fits within byte range
```

28. What are wrapper classes in Java, and how do they relate to primitive types?

Wrapper classes bridge the gap between primitive types and Java's object-oriented nature. Wrapper classes in Java are object representations of the primitive data types.

- They allow primitive values to be treated as objects.
- They enable primitives to interact with collections and provide utility methods.
- Autoboxing and auto-unboxing simplify conversions between primitives and objects.
- Use cases: Collections (e.g., List<Integer> instead of List<int>) and Utility methods (e.g., Integer.parseInt()).

```
int num = 10; // Primitive type
Integer obj = Integer.valueOf(num); // Primitive to Wrapper (Boxing)

int value = obj.intValue(); // Wrapper to Primitive (Unboxing)
```

- Boxing is the process of converting a primitive type to its corresponding wrapper class.
- Unboxing is the process of converting a wrapper class back to its primitive type.
- Wrapper classes allow primitives to be stored in collections.
- Wrapper classes provide methods to perform operations on primitive values.

• Wrapper classes can represent the absence of a value using null, while primitives cannot.

```
ArrayList<Integer> list = new ArrayList<>();
list.add(10); // Autoboxing of int to Integer

String str = "123";
int num = Integer.parseInt(str); // Converts String to int

Integer obj = null; // Allowed
int num = null;     // Compilation Error
```

```
public class WrapperExample {
    public static void main(String[] args) {
        // Primitive type
        int num = 5;

        // Wrapper class (Boxing)
        Integer obj = num; // Autoboxing

        // Using utility method
        String str = obj.toString(); // Converts Integer to String

        // Unboxing
        int value = obj; // Auto-unboxing
        System.out.println("Primitive: " + value);
    }
}
```

29. What are the three types of variables in Java, and how are they different?

Local Variable

A variable declared inside the body of a method. The variable can only be used within that specific method, and it is not accessible outside it.

• Local variables must be initialized before use as they do not have default values.

• They are stored in the stack memory.

• Cannot be declared as static since static implies shared across instances, while local variables are specific to a method's execution.

```
public void display() {
    int localVar = 10;  // Local variable
    System.out.println(localVar);
}
```

Instance Variable

A variable declared inside a class but outside any method or block, and not marked as static. These variables are associated with objects, meaning each object of the class has its own copy of the instance variable.

- Instance variables have default values if not explicitly initialized.
- Stored in the heap memory.
- Their values are instance-specific and not shared among different objects.

```
public class Example {
    int instanceVar;  // Instance variable

    public void show() {
        System.out.println(instanceVar); // Default value: 0
    }
}
```

Static Variable

A variable declared with the static keyword. It is not tied to any particular instance but belongs to the class itself.

- Shared across all objects of the class. Any change to a static variable reflects in all instances of the class.
- Static variables are initialized once when the class is loaded into memory.
- Stored in the method area of memory.
- Can be accessed using the class name directly, without creating an object.

```
public class Example {
    static int staticVar = 100;  // Static variable

    public static void show() {
        System.out.println(staticVar);
    }
}

// Accessing static variable
Example.staticVar = 200; // Accessed directly using the class name
```

30. What is the difference between final, static, and static final variables?

- final: Value cannot be reassigned after initialization.
- static: Shared across all instances; belongs to the class, not any instance.
- static final: A constant value shared across all instances; effectively a class-level constant.

Final Variable

A variable declared with the final keyword is a constant, meaning its value cannot be changed once it has been initialized.

- Must be initialized either at the time of declaration or inside the constructor (for instance variables).
- If the variable holds a reference to an object, the reference cannot be changed, but the object's state can be modified.
- Used to declare constants in a program.

```
public class FinalExample {
    final int MAX_AGE = 100;  // Final variable

    public void display() {
        // MAX_AGE = 101; // Compilation Error: Cannot reassign a final variable
        System.out.println(MAX_AGE);
    }
}
```

Static Variable

A variable declared with the static keyword belongs to the class rather than any specific instance of the class.

- Shared across all instances of the class.
- Memory is allocated for static variables only once, when the class is loaded into memory.
- Can be accessed using the class name directly (e.g., ClassName.variableName).
- Commonly used to store values that are shared among all instances, like configuration values or counters.

```
public class StaticExample {
    static int counter = 0;  // Static variable

    public StaticExample() {
        counter++;  // Increments for every new instance
    }

    public static void display() {
        System.out.println("Counter: " + counter);
    }
}
```

Static Final Variable

A variable that is both static and final is a constant shared across all instances of the class. It must be initialized only once and cannot be changed.

- Combines the properties of static (class-level variable) and final (constant).
- Typically used to define constants that are shared across all instances, like PI, mathematical constants, or configuration values
- The value must be initialized at the time of declaration or inside a static block.

```
public class StaticFinalExample {
    static final double PI = 3.14159;  // Static final variable
```

```
    public static void display() {
        System.out.println("Value of PI: " + PI);
    }
}
```

31. Can you declare a variable without initializing it? And what happens if you try to access an uninitialized local variable?

Yes, for instance and static variables, as they get default values. Local variables must be explicitly initialized before use.

The compiler throws an error if you try to access an uninitialized local variable because local variables do not get default values.

32. What is Object Oriented Programming (OOPs)?

Object-Oriented Programming (OOP) is a programming paradigm based on the concept of "objects," which can contain data in the form of fields (attributes or properties) and code in the form of procedures (methods or functions). OOP focuses on organizing software into objects that interact with each other to accomplish tasks.

OOP promotes modular, maintainable, and scalable software development by organizing code into reusable and understandable components. It emphasizes concepts such as data encapsulation, code reuse, and separation of concerns, which contribute to building robust and flexible software systems.

33. What are the 4 pillars of OOPs?

The four fundamental principles of Object-Oriented Programming (OOP) are commonly referred to as the "Four Pillars of OOP." They are:

Encapsulation

- Encapsulation refers to the bundling of data (attributes or properties) and methods (functions or procedures) that operate on the data into a single unit, called a class.
- It allows the internal state of an object to be hidden from outside interference and accessed only through well-defined interfaces (public methods).
- Encapsulation provides data protection, abstraction, and modularity, enhancing code maintainability and reusability.

Inheritance

- Inheritance is a mechanism that allows a class (subclass or derived class) to inherit properties and behavior (methods) from another class (superclass or base class).
- It promotes code reuse by enabling subclasses to inherit and extend the functionality defined in their superclass.
- Inheritance facilitates the creation of a hierarchy of classes with specialized behavior, promoting code organization and scalability.

Polymorphism

- Polymorphism enables objects to be treated as instances of their superclass, allowing them to take on multiple forms.
- It allows different objects to respond to the same message (method call) in different ways, based on their specific implementations.
- Polymorphism simplifies code by allowing methods to be written to work with objects of a superclass, without needing to know the exact subclass at compile time.
- Polymorphism is achieved through method overriding and method overloading.

Abstraction

- Abstraction involves simplifying complex systems by focusing on the essential properties while hiding unnecessary details.

- It allows developers to create abstract classes and interfaces to define common behavior without specifying implementation details.
- Abstraction helps in managing complexity, improving code maintainability, and promoting code reuse by providing a clear separation between interface and implementation.

These four principles form the foundation of Object-Oriented Programming and guide the design and implementation of software systems, promoting modular, maintainable, and scalable code.

```
class Person {
    // Private data members
    private String name;
    private int age;

    // Constructor
    public Person(String name, int age) {
        this.name = name;
        this.age = age;
    }

    // Public method to access private data
    public String getName() {
        return name;
    }

    public void setName(String name) {
        this.name = name;
    }

    public int getAge() {
        return age;
    }

    public void setAge(int age) {
        if (age > 0) {
            this.age = age;
        }
    }
}

public class EncapsulationExample {
    public static void main(String[] args) {

        //Think of a capsule as a person.
        // The person's name and age are private information,
        // and they have methods to get or update this information safely.
        Person person = new Person("John", 30);
        System.out.println("Name: " + person.getName());
        System.out.println("Age: " + person.getAge());

        person.setAge(35); // updating age
```

```
        System.out.println("Updated Age: " + person.getAge());
    }
}
```

```
//Think of an animal family.
// Dogs and cats both inherit general
// animal characteristics but also
// have their own unique behaviors.
class Animal {
    public void eat() {
        System.out.println("This animal eats food.");
    }
}

class Dog extends Animal {
    public void bark() {
        System.out.println("The dog barks.");
    }
}

public class InheritanceExample {
    public static void main(String[] args) {
        Dog dog = new Dog();
        dog.eat(); // inherited method
        dog.bark(); // specific to Dog class
    }
}
```

```
//Runtime Polymorphism
class Animal {
    void sound() {
        System.out.println("Animal makes a sound");
    }
}

class Dog extends Animal {
    @Override
    void sound() {
        System.out.println("Dog barks");
    }
}

public class Main {
    public static void main(String[] args) {
        Animal animal = new Dog();  // Upcasting
        animal.sound();             // Calls Dog's overridden method
    }
```

```
}
```

```
//Think of an ATM machine.
// You interact with it to withdraw money,
// check your balance, etc.,
// without needing to know how it processes
// transactions internally.
abstract class ATM {
    abstract void withdrawMoney();
    abstract void checkBalance();
}

class MyATM extends ATM {
    @Override
    void withdrawMoney() {
        System.out.println("Money withdrawn.");
    }

    @Override
    void checkBalance() {
        System.out.println("Balance checked.");
    }
}

public class AbstractionExample {
    public static void main(String[] args) {
        ATM atm = new MyATM();
        atm.withdrawMoney(); // Outputs: Money withdrawn.
        atm.checkBalance(); // Outputs: Balance checked.
    }
}
```

34. What is Compile Time Polymorphism and Runtime Polymorphism?

Compile-time polymorphism and runtime polymorphism are two forms of polymorphism in object-oriented programming. They occur at different stages of program execution and are achieved through method overloading and method overriding, respectively.

Compile-Time Polymorphism (Static Binding)

- Compile-time polymorphism occurs when the decision about which method to call is made at compile time based on the method signature.
- It is achieved through method overloading, where multiple methods with the same name but different parameter lists (number or type of parameters) are defined within the same class.
- The compiler determines the appropriate method to invoke based on the number and types of arguments provided in the method call.

• Compile-time polymorphism is also known as static binding because the method call is resolved at compile time and cannot be changed at runtime.

```
class Calculator {
    // Method overloading
    int add(int a, int b) {
        return a + b;
    }

    double add(double a, double b) {
        return a + b;
    }
}

public class Main {
    public static void main(String[] args) {
        Calculator calc = new Calculator();
        int result1 = calc.add(2, 3);          // Calls int add(int a, int b)
        double result2 = calc.add(2.5, 3.5);   // Calls double add(double a, double b)
    }
}
```

Runtime Polymorphism (Dynamic Binding)

• Runtime polymorphism occurs when the decision about which method to call is made at runtime based on the actual type of the object.

• It is achieved through method overriding, where a subclass provides a specific implementation of a method that is already defined in its superclass.

• The JVM determines the appropriate method to invoke based on the actual object type (not the reference type) at runtime.

• Runtime polymorphism allows for flexibility and extensibility in object-oriented designs, as it enables subclass objects to exhibit behavior specific to their type.

```
class Animal {
    void sound() {
        System.out.println("Animal makes a sound");
    }
}

class Dog extends Animal {
    @Override
    void sound() {
        System.out.println("Dog barks");
    }
}

public class Main {
    public static void main(String[] args) {
```

```
        Animal animal = new Dog();  // Upcasting
        animal.sound();             // Calls Dog's overridden method
    }
}
```

In summary, compile-time polymorphism is achieved through method overloading and resolved at compile time, while runtime polymorphism is achieved through method overriding and resolved at runtime based on the actual object type. Each form of polymorphism provides flexibility and allows for code reuse in object-oriented programming.

35. What are the different types of inheritance?

Inheritance is a fundamental concept in object-oriented programming that allows a class (subclass or derived class) to inherit properties and behavior (methods) from another class (superclass or base class). There are several types of inheritance, based on the relationship between the subclass and superclass:

Single Inheritance

- Single inheritance occurs when a subclass inherits from only one superclass.
- It is the simplest form of inheritance and is widely supported in most object-oriented programming languages.

```
class Animal {
    // Animal properties and methods
}

class Dog extends Animal {
    // Dog inherits from Animal
}
```

Multilevel Inheritance

- Multilevel inheritance occurs when a subclass inherits from another subclass, forming a chain of inheritance.
- It allows for the creation of a hierarchy of classes with multiple levels of specialization.

```
class Animal {
    // Animal properties and methods
}

class Mammal extends Animal {
    // Mammal inherits from Animal
}

class Dog extends Mammal {
    // Dog inherits from Mammal
```

```
}
```

Hierarchical Inheritance

- Hierarchical inheritance occurs when multiple subclasses inherit from the same superclass.
- It allows for the creation of a hierarchy of classes with different levels of specialization, branching out from a common superclass.

```
class Animal {
    // Animal properties and methods
}

class Dog extends Animal {
    // Dog inherits from Animal
}

class Cat extends Animal {
    // Cat inherits from Animal
}
```

Multiple Inheritance via Interfaces in Java

- Multiple inheritance occurs when a subclass inherits from multiple superclasses.
- While Java does not support multiple inheritance of classes due to the diamond problem, it supports multiple inheritance through interfaces.
- Interfaces in Java allow a class to inherit abstract methods from multiple interfaces, enabling a form of multiple inheritance.

```
interface Swimmer {
    void swim();
}

interface Flyer {
    void fly();
}

class Bird implements Swimmer, Flyer {
    // Bird implements both Swimmer and Flyer interfaces
}
```

36. What is diamond problem in Java?

The diamond problem occurs when a class inherits from two or more classes, each of which inherits from the same superclass. This creates a diamond-shaped inheritance hierarchy, where a subclass inherits the same superclass through multiple paths.

The problem arises when the subclass attempts to inherit or override a method from the common superclass. In such cases, the compiler may not be able to determine which version of the method to use, leading to ambiguity and potential conflicts.

```
    A
   / \
  B   C
   \ /
    D
```

In this diamond-shaped inheritance hierarchy, both classes B and C inherit from class A, and class D inherits from both B and C. If class A has a method foo(), and classes B and C override foo() with different implementations, the compiler may not be able to determine which version of foo() to use in class D.

In Java, the diamond problem is avoided by allowing single inheritance of classes and supporting multiple inheritance only through interfaces. Interfaces do not contain any implementation, so there is no ambiguity in method resolution.

37. What is difference between abstract class and interface?

Abstract classes and interfaces are both used to achieve abstraction and define contracts for classes in Java, but they have some key differences:

Abstract Class

- An abstract class is a class that cannot be instantiated on its own and may contain abstract methods (methods without implementation) as well as concrete methods (methods with implementation).
- Abstract classes can have constructors, instance variables, and non-abstract methods, in addition to abstract methods.
- A subclass must provide implementations for all abstract methods defined in its abstract superclass, or it must itself be declared abstract.
- Abstract classes are used to define common behavior and characteristics shared among subclasses, serving as a blueprint for creating concrete subclasses.

```
abstract class Shape {
    // Abstract method
    abstract double area();

    // Concrete method
    void display() {
        System.out.println("Displaying shape");
    }
}
```

Interface

- An interface is a reference type similar to a class but contains only abstract methods, constants (static

final fields), and default/static methods (methods with default implementations).

• Interfaces cannot have constructors or instance variables, only method signatures.

• A class implements an interface by providing concrete implementations for all the methods declared in the interface.

• Interfaces are used to define contracts that classes can choose to implement, enabling multiple inheritance of type (a class can implement multiple interfaces).

```
interface Drawable {
    // Abstract method
    void draw();
}
```

Key Differences

• An abstract class can have both abstract and concrete methods, while an interface can have abstract methods, default methods, and static methods.

• A class can extend only one abstract class, but it can implement multiple interfaces.

• Abstract classes can have constructors, whereas interfaces cannot.

• Abstract classes are used to provide a common base for subclasses to extend, while interfaces define contracts that classes can implement to achieve polymorphism.

```
// Abstract class
abstract class Animal {
    private String name;

    // Constructor in abstract class
    public Animal(String name) {
        this.name = name;
    }

    // Abstract method (no implementation)
    public abstract void makeSound();

    // Concrete method
    public void sleep() {
        System.out.println(name + " is sleeping.");
    }
}

// Interface with abstract methods and default method
interface Pet {
    // Abstract method
    void play();

    // Default method (concrete method)
    default void beFriendly() {
        System.out.println("Being friendly.");
```

```
    }

    // Static method (concrete method)
    static void showAffection() {
        System.out.println("Showing affection.");
    }
}

// Class that extends abstract class and implements interface
class Dog extends Animal implements Pet {
    public Dog(String name) {
        super(name); // Call the constructor of the abstract class
    }

    @Override
    public void makeSound() {
        System.out.println("Dog barks.");
    }

    @Override
    public void play() {
        System.out.println("Dog is playing.");
    }
}


public class AbstractVSInterfaceExample {
    public static void main(String[] args) {
        Dog myDog = new Dog("Buddy");
        myDog.makeSound(); // Dog barks.
        myDog.sleep(); // Buddy is sleeping.
        myDog.play(); // Dog is playing.
        myDog.beFriendly(); // Being friendly.

        Pet.showAffection(); // Showing affection.
    }
}
```

38. When to use abstract class VS when to use interface?

Use Interface When

A is capable of [doing this] scenario fits in your requirement. For example: below Programmable interface is capable of doing Programming

```
interface Programmable {
        void programming();
}
```

• You want to define a contract for classes to implement: Interfaces specify a set of methods that implementing classes must provide, defining a clear contract for how classes interact with each other.

• You want to achieve multiple inheritance of type: Classes can implement multiple interfaces, allowing them to inherit behavior from multiple sources without the restrictions of single inheritance.

• You want to decouple implementation from interface: Interfaces provide a way to separate the specification of behavior from its implementation, promoting loose coupling and flexibility in design.

Use Abstract Class When

• A is B or C is B scenario fits in your requirement, for example: Dog is Animal, Cat is Animal..etc.

• You want to provide a common base implementation for subclasses: Abstract classes can contain both abstract and concrete methods, allowing you to define common behavior that subclasses can inherit.

• You want to define a class hierarchy: Abstract classes are useful for creating a hierarchical structure of classes where each subclass represents a more specialized version of the superclass.

• You want to share code among closely related classes: Abstract classes can contain state and behavior that is common to multiple subclasses, reducing code duplication.

39. What is Constructor and different types of Constructors?

A constructor in Java is a special type of method that is automatically called when an instance (object) of a class is created. It is used to initialize the newly created object and set its initial state. Constructors have the same name as the class and do not have a return type, not even void.

There are several types of constructors in Java:

Default Constructor

• A default constructor is a non-parameterized constructor automatically provided by the Java compiler if no other constructor is explicitly defined in a class.

• It initializes the object with default values (e.g., numeric fields are initialized to 0, references are initialized to null).

```
public class MyClass {
    // Default constructor
    public MyClass() {
        // Constructor body
    }
}
```

Parameterized Constructor

• A parameterized constructor is a constructor with parameters that allows you to initialize the object with specific values passed as arguments.

• It enables you to customize the initialization of objects based on the values provided at the time of object creation.

```
public class MyClass {
    // Parameterized constructor
    public MyClass(int x, int y) {
        // Constructor body
    }
}
```

Copy Constructor

- A copy constructor is a constructor that initializes an object by copying the state of another object of the same class.
- It allows you to create a new object with the same state as an existing object, providing a convenient way to clone objects.

```
public class MyClass {
    private int x;

    // Copy constructor
    public MyClass(MyClass obj) {
        this.x = obj.x;  // Copy state from obj
    }
}
```

40. Is there a destructor concept in Java?

No, Java does not have a destructor concept like some other programming languages such as C++. In Java, objects are automatically garbage collected by the JVM (Java Virtual Machine) when they are no longer referenced or needed.

There was finalize() method as a workaround for the java destructor but it is also deprecated since Java 9.

41. What are different Types of Loops in Java?

In Java, there are several types of loops that allow you to execute a block of code repeatedly until a certain condition is met. The main types of loops are:

for loop

- The for loop is used when you know in advance how many times you want to execute a block of code.
- It consists of three parts: initialization, condition, and increment/decrement.

```
for (int i = 0; i < 5; i++) {
    System.out.println("Iteration " + i);
}
/*Output
        Iteration 0
        Iteration 1
        Iteration 2
        Iteration 3
        Iteration 4*/
```

while loop

- The while loop is used when you want to execute a block of code repeatedly as long as a condition is true.
- It only consists of a condition, and the loop continues to execute as long as the condition evaluates to true.

```
int i = 0;
while (i < 5) {
    System.out.println("Iteration " + i);
    i++;
}
/*Output
        Iteration 0
        Iteration 1
        Iteration 2
        Iteration 3
        Iteration 4*/
```

do-while loop

- The do-while loop is similar to the while loop, but it guarantees that the block of code is executed at least once before checking the condition.
- It consists of a block of code followed by a while condition.

```
int i = 0;
do {
    System.out.println("Iteration " + i);
    i++;
```

```
} while (i < 5);
/*Output
        Iteration 0
        Iteration 1
        Iteration 2
        Iteration 3
        Iteration 4*/
```

enhanced for loop (for-each loop)

- The enhanced for loop (also known as the for-each loop) is used to iterate over elements of arrays or collections.
- It simplifies iterating through arrays or collections by hiding the details of the iterator.

```
int[] numbers = {1, 2, 3, 4, 5};
for (int num : numbers) {
    System.out.println(num);
}
/*Output
        1
        2
        3
        4
        5*/
```

42. How Can You Exit From a Loop?

Using the break statement

- The break statement is used to terminate the loop immediately and transfer control to the statement following the loop.
- It can be used with for, while, do-while, and switch statements.

```
for (int i = 0; i < 10; i++) {
    if (i == 5) {
        break; // Exit loop when i equals 5
    }
    System.out.println(i);
}
```

Using the return statement

- If the loop is contained within a method, you can use the return statement to exit the method and the loop simultaneously.
- This terminates the loop and returns control to the caller of the method.

```
public void someMethod() {
    for (int i = 0; i < 10; i++) {
        if (i == 5) {
            return; // Exit method and loop when i equals 5
        }
        System.out.println(i);
    }
}
```

Using the continue statement

- The continue statement is used to skip the rest of the current iteration and continue with the next iteration of the loop.
- It can be used with for, while, and do-while loops.

```
for (int i = 0; i < 10; i++) {
    if (i % 2 == 0) {
        continue; // Skip even numbers
    }
    System.out.println(i);
}
```

43. When to switch statement vs if-else statement?

Use a switch statement when:

- Multiple Conditions: You have multiple conditions to evaluate based on the value of a single expression or variable.
- Fixed Values: The conditions involve fixed, integral values (e.g., integers, characters, enums). switch statements are optimized for such scenarios.
- Readability: Using a switch statement can make the code more readable, especially when dealing with a large number of conditions.

```
switch (dayOfWeek) {
    case 1:
        System.out.println("Monday");
        break;
    case 2:
        System.out.println("Tuesday");
        break;
    // More cases...
    default:
        System.out.println("Invalid day");
}
```

Use a if-else statement when:

- Complex Conditions: You have complex conditions that cannot be easily expressed using a switch statement.
- Non-fixed Values: The conditions involve non-fixed values or expressions that cannot be directly compared to each other.
- Chained Conditions: You have a series of conditions where the evaluation of one condition depends on the result of the previous one.

```
if (age >= 18 && age < 65) {
    System.out.println("Adult");
} else if (age >= 65) {
    System.out.println("Senior Citizen");
} else {
    System.out.println("Minor");
}
```

44. How try/catch/finally works in Java?

In Java, the try, catch, and finally blocks are used for exception handling, allowing you to gracefully manage errors that might occur during the execution of your code.

Here's how they work:

- try: This block encloses the code that you want to monitor for exceptions. If an exception occurs within this block, it is thrown.
- catch: This block catches exceptions that are thrown within the corresponding try block. You can specify which type of exception you want to catch by specifying the exception type in the catch block. You can have multiple catch blocks to handle different types of exceptions.
- finally: This block, if present, is executed regardless of whether an exception is thrown or not. It's typically used to perform cleanup tasks, like closing file handles or releasing other resources, that should always occur, whether an exception is thrown or not.

```
try {
    // Code that might throw an exception
    int result = divide(10, 0); // This will throw an ArithmeticException
} catch (ArithmeticException e) {
    // Handle the specific exception
    System.out.println("An arithmetic exception occurred: " + e.getMessage());
} finally {
    // This block will always execute, regardless of whether an exception is caught or not
    System.out.println("Finally block executed.");
}
```

45. What are the scenarios in which finally block will not get executed?

In Java, the finally block is designed to execute regardless of whether an exception is thrown or not. However, there are a few scenarios in which the finally block may not get executed:

System.exit()

If the JVM terminates abruptly due to a call to System.exit(), the finally block will not get executed. This is because System.exit() immediately terminates the JVM and does not allow any further code execution.

Infinite loop

If the try or catch block contains an infinite loop or some other code that prevents control from reaching the finally block, then the finally block will not be executed until the loop terminates or control is transferred out of the block.

Fatal errors

If a fatal error occurs, such as a StackOverflowError or an OutOfMemoryError, the JVM may terminate abruptly without executing the finally block.

In all other scenarios, the finally block should execute as expected, even if an exception is thrown or caught within the try block.

46. How to use try-with-resources in Java?

Try-with-resources is a feature for automatic resource management. It's used to automatically close resources that implement the AutoCloseable interface, such as streams, database connections, or sockets, at the end of a try block.

In below code example, the FileReader is automatically closed at the end of the try block, even if an exception is thrown while reading from the file.

```
try (FileReader fileReader = new FileReader("example.txt")) {
    // Code that reads from the file
    int data = fileReader.read();
    while (data != -1) {
        System.out.print((char) data);
        data = fileReader.read();
    }
} catch (IOException e) {
    // Exception handling
    e.printStackTrace();
}
```

47. What Is the Difference Between a Checked and an Unchecked Exception?

In Java, exceptions are categorized into two main types: checked exceptions and unchecked exceptions. The main differences between them are related to how they are handled by the compiler and how they affect the

code:

Checked Exceptions

- Checked exceptions are subclasses of Exception (excluding RuntimeException and its subclasses).
- They must be either caught or declared to be thrown by the method using the throws keyword.
- Examples of checked exceptions include IOException, ClassNotFoundException, and SQLException.
- The compiler enforces handling of checked exceptions, which means that you must either catch them or declare that your method throws them. This helps to ensure that developers are aware of potential exceptional conditions and take appropriate action.

```
File file = new File("example.txt");
        try {
            FileReader fileReader = new FileReader(file);
            // Code that reads from the file
        } catch (IOException e) {
            // Handle the checked exception
            e.printStackTrace();
        }
```

Unchecked Exceptions (RuntimeExceptions)

- Unchecked exceptions are subclasses of RuntimeException.
- They do not need to be explicitly caught or declared by the method using the throws keyword.
- Examples of unchecked exceptions include NullPointerException, ArrayIndexOutOfBoundsException, and ArithmeticException.
- The compiler does not enforce handling of unchecked exceptions. This means that it's up to the developer to write code to handle or prevent them if necessary, but it's not mandatory to do so.
- Unchecked exceptions often represent programming errors or unexpected conditions that may occur at runtime, rather than conditions that can be reasonably anticipated and handled by the code.

```
String str = null;
        try {
            int length = str.length(); // This will throw a NullPointerException
        } catch (NullPointerException e) {
            // Handle the unchecked exception
            System.out.println("NullPointerException occurred: " + e.getMessage());
        }
```

In summary, checked exceptions are enforced by the compiler, must be handled explicitly, and typically represent anticipated exceptional conditions related to external factors (like file I/O or network operations). Unchecked exceptions, on the other hand, do not need to be handled explicitly and often represent programming errors or unexpected runtime conditions.

48. What Is the Difference Between an Unlabeled and a Labeled break Statement?

Unlabeled break Statement

- It only breaks out of the innermost loop or switch statement in which it appears.
- It is typically used in for, while, do-while, or switch statements.

Labeled break Statement

- It allows you to break out of a specific, outer loop or block of code.
- This is useful when dealing with nested loops or blocks, where you want to break out of a loop that is not the innermost one.

```
for (int i = 0; i < 5; i++) {
           for (int j = 0; j < 3; j++) {
               if (j == 2) {
                   System.out.println(j);
                   break; // Breaks out of the inner loop
               }
           }
           System.out.println(i);
       }

       System.out.println("Outer Loop");

       outerloop:
       for (int i = 0; i < 3; i++) {
           for (int j = 0; j < 5; j++) {
               System.out.println(i);
               System.out.println(j);
               if (i == 1 && j == 2) {
                   break outerloop; // Breaks out of the outer loop
               }
           }
       }
```

In summary, unlabeled breaks are used to exit the current loop or switch, while labeled breaks provide a way to exit from multiple nested loops or switches, though their use is generally discouraged in favor of clearer code structures.

49. What Is the Use of the Throw and Throws Keywords?

In Java, both throw and throws are related to exception handling, but they serve different purposes:

throw Keyword

- The throw keyword is used to explicitly throw an exception within a method or block of code.

- It's typically used when a particular condition occurs that warrants an exception to be thrown, either because of an error condition or because of some other exceptional circumstance.
- You can throw both checked and unchecked exceptions using the throw keyword.

Below is the code example:

```
public void validateAge(int age) {
    if (age < 0) {
        throw new IllegalArgumentException("Age cannot be negative");
    }
    // Other validation logic
}
```

throws Keyword

- The throws keyword is used in a method declaration to indicate that the method may throw certain types of exceptions.
- It's used to delegate the responsibility of handling exceptions to the caller of the method.
- When a method declares that it throws an exception using the throws keyword, it's informing the compiler that the method may throw that type of exception, and the calling code must either catch the exception or declare it to be thrown further up the call stack.
- Multiple exceptions can be declared in the throws clause, separated by commas.

```
public void readFile(String fileName) throws IOException, FileNotFoundException {
    // Code that reads from the file
}
```

In summary, throw is used to explicitly throw an exception within a method, while throws is used in the method signature to indicate that the method may throw certain types of exceptions, thereby shifting the responsibility of handling those exceptions to the caller of the method.

50. How to Catch Multiple Exceptions?

1. Using Multiple Catch Blocks

You can have multiple catch blocks, each handling a different type of exception:

```
try {
    // Code that may throw exceptions
} catch (IOException e) {
    // Handle IOException
} catch (SQLException e) {
    // Handle SQLException
} catch (Exception e) {
    // Handle other types of exceptions
}
```

2. Using Multi-Catch Blocks

You can use a single catch block to catch multiple exceptions if they have a common superclass or interface. This approach is useful when you want to handle exceptions in a similar way:

```
try {
    // Code that may throw exceptions
} catch (IOException | SQLException e) {
    // Handle IOException and SQLException
} catch (Exception e) {
    // Handle other types of exceptions
}
```

51. What Is the Difference Between an Error and Exception?

In Java, both errors and exceptions are subclasses of the Throwable class, but they serve different purposes and are meant to be handled differently:

Exception

- Exceptions represent exceptional conditions that occur during the execution of a program. They are typically caused by problems that are outside the control of the program, such as invalid input, network issues, or file I/O errors.
- Exceptions can be checked or unchecked. Checked exceptions must be either caught or declared in the method signature using the throws keyword, while unchecked exceptions do not need to be caught or declared.
- Examples of exceptions include IOException, NullPointerException, ArrayIndexOutOfBoundsException, etc.

Error

- Errors represent abnormal conditions that occur in the Java Virtual Machine (JVM) or in the runtime environment, which are typically not recoverable by the application itself.
- Errors are typically caused by serious problems that are beyond the control of the programmer, such as system failures, out-of-memory errors, or stack overflow errors.
- Errors are unchecked, which means they do not need to be caught or declared by the programmer. In fact, it's generally not recommended to catch errors because they often indicate serious problems that the application cannot recover from.
- Examples of errors include OutOfMemoryError, StackOverflowError, VirtualMachineError, etc.

In summary, exceptions are meant to be caught and handled within the program logic to gracefully recover from exceptional conditions, while errors indicate serious problems at the system level that may not be recoverable by the application itself. Handling errors is generally not recommended because they often indicate critical issues that may require intervention at a higher level.

52. How can you create a custom exception in Java?

To create a custom exception in Java, you need to create a new class that extends either Exception (for checked exceptions) or RuntimeException (for unchecked exceptions).

1. Define the Custom Exception Class

- Create a new class that extends Exception for a checked exception or RuntimeException for an unchecked exception.
- Provide constructors to initialize the exception.

2. Add Constructors

- Include at least one constructor that takes a String message to describe the error.
- Optionally, add constructors that take a Throwable cause and both a message and a cause.

```
//example of creating a custom checked exception
// Define the custom exception
public class CustomCheckedException extends Exception {
    // Constructor that accepts a message
    public CustomCheckedException(String message) {
        super(message);
    }

    // Constructor that accepts a message and a cause
    public CustomCheckedException(String message, Throwable cause) {
        super(message, cause);
    }

    // Constructor that accepts a cause
    public CustomCheckedException(Throwable cause) {
        super(cause);
    }
}
```

```
// example of creating a custom unchecked exception
//Define the custom exception
public class CustomUncheckedException extends RuntimeException {
    // Constructor that accepts a message
    public CustomUncheckedException(String message) {
        super(message);
    }

    // Constructor that accepts a message and a cause
    public CustomUncheckedException(String message, Throwable cause) {
        super(message, cause);
    }

    // Constructor that accepts a cause
```

```
    public CustomUncheckedException(Throwable cause) {
        super(cause);
    }
}
```

53. Can you have a try block without a catch block in Java?

Yes, you can have a try block without a catch block in Java. This is valid syntax and is often used when you want to handle exceptions at a higher level or simply when you want to execute some code that might throw an exception, without handling it immediately.

In such cases, you must follow the try block with either a catch block or a finally block, or both. Here are a few scenarios:

try-finally

You can have a try block followed by a finally block without a catch block. This ensures that the finally block is executed regardless of whether an exception is thrown or not.

```
try {
    // Code that may throw exceptions
} finally {
    // Code that always executes, whether exception occurs or not
}
```

try-with-resources

You can use try-with-resources statement to automatically close resources that implement the AutoCloseable interface. This also allows a try block without a catch block.

```
static String readFirstLineFromFile(String path) throws IOException {
    try (BufferedReader br = new BufferedReader(new FileReader(path))) {
        return br.readLine();
    }
}
```

54. Explain the difference between StackOverflowError and OutOfMemoryError in Java?

The main difference between StackOverflowError and OutOfMemoryError is their underlying causes and contexts:

- StackOverflowError occurs due to excessive stack usage, often caused by deep recursion or method call chains.
- OutOfMemoryError occurs due to insufficient heap memory, often caused by excessive object creation,

memory leaks, or large data processing tasks.

Both errors indicate serious issues in the application, and they require investigation and resolution to ensure the stability and reliability of the Java application.

55. What is Java Garbage Collection and how does it work?

Java Garbage Collection (GC) is the process by which Java programs perform automatic memory management. The Java runtime environment deletes objects when they are no longer in use to free up memory. This is done automatically by the garbage collector, which is part of the Java Virtual Machine (JVM).

GC is essential in long-running server applications to prevent memory leaks and ensure efficient memory usage.

56. What are the different garbage collection algorithms in Java?

Java provides several garbage collection algorithms:

- Serial GC: Uses a single thread to perform all garbage collection work, suitable for single-threaded applications.
- Parallel GC: Uses multiple threads for garbage collection, suitable for multi-threaded applications.
- CMS (Concurrent Mark-Sweep) GC: Performs most of the garbage collection concurrently with the application threads to minimize pause times.
- G1 (Garbage-First) GC: Divides the heap into regions and prioritizes reclaiming regions with the most garbage.

G1 GC is often used in large applications where predictability and low pause times are critical.

57. What is a memory leak in Java, and how can you detect it?

Memory leak in Java occurs when objects are no longer needed but are still referenced, preventing the garbage collector from reclaiming their memory. Tools like VisualVM, JProfiler, or YourKit can be used to detect memory leaks by monitoring heap usage and identifying objects that are not being garbage collected.

It is essential to identify and address memory leaks in high-availability systems to prevent them from gradually consuming memory resources. This proactive approach ensures that these systems do not experience memory exhaustion over time, which could potentially lead to performance degradation or system failures.

By leveraging tools like VisualVM, JProfiler, or YourKit, developers can effectively monitor heap usage and pinpoint any lingering objects that are impeding proper garbage collection. Taking prompt action to fix memory leaks is paramount for maintaining the stability and efficiency of such systems in the long run.

58. What are Soft, Weak, and Phantom references in Java?

SoftReference: Objects stored in SoftReference instances are cleared at the discretion of the garbage collector in response to memory demand. This means that when the system is running low on memory, objects held only by SoftReferences may be reclaimed to free up space for other data.

WeakReference: Objects stored in WeakReference instances are cleared as soon as they are no longer strongly reachable. This happens when there are no more strong references pointing to the object, making it eligible for garbage collection. WeakReferences are commonly used for implementing memory-sensitive caches where entries can be easily removed to prevent excessive memory consumption.

PhantomReference: Objects stored in PhantomReference instances are cleared after the garbage collector determines that they are no longer reachable. However, unlike WeakReferences, objects held by PhantomReferences are not immediately reclaimed when they become unreachable. Instead, they are enqueued for cleanup actions, allowing for additional processing before their final removal.

WeakReferences are particularly useful in scenarios such as caching mechanisms where objects can be discarded if memory resources are needed elsewhere.

This flexibility in managing memory usage efficiently makes WeakReferences a valuable tool in optimizing memory utilization in Java applications.

```
        // Strong Reference (Default)
        // Not collected
        Object strongRef = new Object();

        // Weak Reference
        //collected by the garbage collector if there are
        // no strong references to it. (Caching)
        WeakReference<Object> weakRef = new WeakReference<>(new Object());

        // Soft Reference (used in image caching libraries )
        //collected by the garbage collector if memory pressure increases.
        SoftReference<Object> softRef = new SoftReference<>(new Object());

        // Phantom Reference (Memory Management Frameworks)
        // weakest type of reference and
        // are mainly used to track when an object is being
        // finalized by the garbage collector.
        ReferenceQueue<Object> phantomQueue = new ReferenceQueue<>();
        PhantomReference<Object> phantomRef = new PhantomReference<>(new Object(), phantomQueue);

        // Let's simulate some memory pressure
        System.gc();

        // Print status of references
        System.out.println("Strong reference: " + strongRef); //java.lang.Object@5acf9800
        System.out.println("Weak reference: " + weakRef.get()); //null
        System.out.println("Soft reference: " + softRef.get()); //java.lang.Object@4617c264
        System.out.println("Phantom reference: " + phantomRef.get()); //null
```

59. How can you force garbage collection in Java?

You can suggest the JVM to perform garbage collection using System.gc() or Runtime.getRuntime().gc(). However, it's important to note that there's no guarantee that the garbage collector will run immediately upon invocation. Forcing GC may be utilized in testing scenarios to observe and analyze the garbage collection behavior in specific situations.

```
public static void main(String[] args) {
       Object obj = new Object();
       obj = null;
       System.gc();
   }
```

60. What are the common JVM options for tuning garbage collection?

JVM options for tuning GC include:

- -Xms and -Xmx for setting initial and maximum heap size.
- -XX:+UseG1GC to enable G1 Garbage Collector.
- -XX:MaxGCPauseMillis to set the maximum pause time goal for G1 GC.
- -XX:+PrintGCDetails to print detailed GC logs.

Tuning GC is essential for optimizing application performance, especially in large-scale applications.

61. Explain the internal Working of Java's G1 Garbage Collector and When to Use G1 GC?

Below are the key Concepts for G1 Garbage Collector:

- Regions: G1 divides the heap into numerous fixed-size regions (usually 1MB to 32MB). These regions are not limited to a specific generation (young or old).
- Remembered Sets (RSets): Each region has an associated RSet that tracks references from other regions pointing into it. This eliminates the need for full heap scans during garbage collection.
- Collection Set (CSet): The set of regions selected for garbage collection in a given cycle. G1 aims to collect regions with the most garbage first.
- Pause Time Goal: G1 GC prioritizes meeting user-defined pause time goals, making it suitable for applications with low latency requirements.

Collection Phases

1. Young Collection:

- Marking: G1 marks live objects starting from the roots (GC roots) using concurrent marking.
- Evacuation: Live objects in young regions are copied (evacuated) to survivor or old regions. This phase is done in parallel using multiple threads.
- Reclamation: Empty young regions are immediately made available for allocation.

2. Mixed Collection (Concurrent and Parallel Phases):

- Initial Marking (STW): Briefly pauses the application to mark the roots.
- Concurrent Marking: G1 concurrently traces live objects.
- Remark (STW): Finalizes the marking process by revisiting any objects that were modified during the

concurrent phase.

• Cleanup (Mostly STW): Identifies completely empty regions (candidates for immediate reclamation) and performs accounting related to liveness information.

• Copying/Evacuation (Parallel): Selects a set of old regions (based on the pause time goal) and copies live objects to new regions, freeing up the old regions.

Other Features

• Humongous Objects: Large objects (larger than half a region) are handled specially in "humongous regions."

• String Deduplication: G1 can optionally eliminate duplicate strings in the heap to save space.

• Adaptive Behavior: G1 GC continuously adapts its behavior based on application characteristics and pause time goals.

How G1 Achieves Its Goals

• Predictable Pauses: By collecting regions in smaller increments and focusing on the most garbage-filled regions, G1 GC aims to meet pause time goals.

• High Throughput: The parallel and concurrent nature of many phases allows for efficient garbage collection.

• Good Heap Utilization: The region-based approach helps with fragmentation issues.

When to Use G1 GC

• Large Heaps: G1 GC is designed for multi-gigabyte heaps.

• Low Latency Requirements: If your application needs to meet strict pause time goals.

• Mixed Workloads: G1 GC handles both young and old generation objects well.

62. How does the garbage collector know which objects are reachable?

The concept of reachability analysis in garbage collection is a fundamental technique used to manage memory in programming languages. When the garbage collector executes reachability analysis, it starts from certain designated root objects, such as local variables or static variables.

These root objects are essentially the starting points from which the garbage collector explores and traverses through the entire object graph to identify all reachable objects. As the garbage collector follows references from the root objects to other objects in memory, it marks each visited object as reachable.

Objects that are not encountered during this traversal process are considered unreachable or detached from the root objects. These unreachable objects are deemed unnecessary and can be safely removed from memory to free up resources. By conducting reachability analysis, the garbage collector can effectively distinguish between active objects that are still in use within the program and obsolete objects that can be safely discarded.

This proactive memory management strategy helps prevent memory leaks and ensures efficient utilization of memory resources in software applications.

63. What are some of the Useful Annotations used in Java?

Annotations in Java are like metadata tags that provide additional information about your code to the compiler or runtime environment. They're not part of the program's logic but offer instructions or hints.

Built-in Annotations

- @Override: Indicates that a method in a subclass is intended to override a method in its superclass. Helps prevent errors by ensuring the signature matches.
- @Deprecated: Marks a class, method, or field as deprecated, meaning it's discouraged from being used (often because it's obsolete or has a better alternative).
- @SuppressWarnings: Instructs the compiler to suppress specific warnings for the annotated element.
- @FunctionalInterface: Indicates that an interface is intended to be a functional interface (having a single abstract method) to be used with lambda expressions.
- @SafeVarargs: Used on a method or constructor that takes a variable number of arguments (varargs) to suppress warnings related to heap pollution (when a varargs parameter's array might contain objects of an incorrect type).

```
@Override
public String toString() { // Overrides the toString method
    return "This is my custom object";
}

@Deprecated
public void oldMethod() {  // Discourages use of this method
    // ...
}
```

64. What Are Meta-Annotations?

Think of meta-annotations as annotations that apply to other annotations. They provide information about how other annotations should behave or where they can be used. Here's a breakdown of the key meta-annotations:

@Retention

- Specifies how long an annotation should be retained in the code.
- RetentionPolicy.SOURCE: Discarded by the compiler.
- RetentionPolicy.CLASS: Retained in the compiled class files but not available at runtime.
- RetentionPolicy.RUNTIME: Retained at runtime and accessible through reflection.

@Target

- Specifies the types of elements (e.g., classes, methods, fields, parameters) that an annotation can be applied to.
- ElementType.TYPE: Classes, interfaces, enums.

- ElementType.METHOD: Methods.
- ElementType.FIELD: Fields (class variables).
- ElementType.PARAMETER: Method parameters.
- ElementType.CONSTRUCTOR: Constructors.

@Documented

Indicates that an annotation should be included in the Javadoc documentation for the annotated element.

@Inherited

Allows an annotation on a class to be inherited by its subclasses.

```
@Retention(RetentionPolicy.RUNTIME) // Retained at runtime
@Target(ElementType.METHOD)         // Can be applied to methods
public @interface MyAnnotation {
    // ... annotation elements
}
```

65. What Are Repeating Annotations?

Before Java 8, you could apply an annotation only once to an element. Repeating annotations (introduced in Java 8) allow you to apply the same annotation multiple times to the same element. Here's how:

Create a Container Annotation

- This annotation acts as a container to hold multiple instances of the repeating annotation.
- Annotate it with @Repeatable, specifying the repeating annotation's class.

Create the Repeating Annotation

This is the annotation you want to apply multiple times.

```
@Repeatable(Authors.class)
public @interface Author {
    String name();
}

public @interface Authors {
    Author[] value();
}


//Now you can use @Author multiple times
@Author(name = "John Doe")
@Author(name = "Jane Smith")
public class MyClass {
    // ...
}
```

Key Points

- Meta-annotations control the behavior and usage of other annotations.
- Repeating annotations allow for applying the same annotation multiple times, increasing flexibility.
- Both features enhance the expressiveness and capabilities of Java annotations.

66. Is It Possible to Extend Annotations and what is Indirect Extension?

You cannot directly extend an annotation in Java using the extends keyword, as you would with classes or interfaces. Annotations are a distinct type of metadata, not classes.

Indirect Extension

- You can achieve a kind of "inheritance" effect with annotations by combining them.
- Meta-annotations like @Inherited: If an annotation is marked with @Inherited, and you apply it to a class, then subclasses of that class will automatically inherit the annotation.
- Custom composed annotations: You can create a new annotation that combines multiple existing annotations. This way, applying your custom annotation is equivalent to applying all the combined annotations.

In below example, MyCustomAnnotation indirectly "extends" the Author annotation, as applying MyCustomAnnotation also applies Author.

So you can create custom annotations that combine multiple existing annotations, effectively achieving an "extension" effect.

```
@Retention(RetentionPolicy.RUNTIME)
@Target(ElementType.TYPE)
public @interface MyCustomAnnotation {
    Author author(); // Uses the previously defined @Author annotation
    int version() default 1;
}
```

```
static String readFirstLineFromFile(String path) throws IOException {
    try (BufferedReader br = new BufferedReader(new FileReader(path))) {
        return br.readLine();
    }
}
```

67. How to Limit the Elements in Which an Annotation Can Be Applied?

You can control the elements where an annotation can be applied using the @Target meta-annotation. Here's how it works:

The @Target Meta-Annotation

- @Target accepts an array of ElementType values, which specify the types of elements the annotation can

be used on.

Common ElementType values include:

- ElementType.TYPE: Classes, interfaces, enums.
- ElementType.METHOD: Methods.
- ElementType.FIELD: Fields (class variables).
- ElementType.PARAMETER: Method parameters.
- ElementType.CONSTRUCTOR: Constructors.
- ElementType.LOCAL_VARIABLE: Local variables.
- ElementType.ANNOTATION_TYPE: Annotations (for creating meta-annotations).
- ElementType.PACKAGE: Package declarations.
- ElementType.TYPE_PARAMETER: Type parameters (generics).
- ElementType.TYPE_USE: Type uses (introduced in Java 8).

```
import java.lang.annotation.*;

@Retention(RetentionPolicy.RUNTIME)
@Target(ElementType.METHOD) // This annotation can only be applied to methods
public @interface LogExecutionTime {
    // ... annotation elements (if any)
}
```

68. What is the Difference Between Primitive and Reference Types?

In programming, particularly in languages like Java, C#, and JavaScript, data types are broadly categorized into primitive types and reference types. Understanding the differences between these two is crucial for effective coding and memory management.

Primitive Types

Primitive types are the most basic data types provided by a programming language. They are:

- Simple and Immutable: They hold simple values and are immutable, meaning once a primitive value is created, it cannot be altered.
- Stored Directly: The actual value is stored directly in memory locations assigned to the variable.
- Fixed Size: They typically have a fixed size which can be directly represented in the memory. This size can vary between programming languages.

eg: int, char, boolean, byte, short, long, float, double

Reference Types

Reference types are more complex. They include structures that reference or point to memory locations where the actual data is stored.

- Contain References: Instead of holding the actual value, these types hold a reference (or address) to the memory location where the data is stored.
- Mutable: Generally, the objects these references point to can be modified.
- Dynamic Size: The size can be dynamic and not necessarily fixed since it can point to complex data structures like objects and arrays.

eg: All objects (instances of classes), arrays

Key Differences

Memory Allocation:

- Primitive Types: Stored directly in the stack. The memory allocation is straightforward and typically faster.
- Reference Types: Stored in the heap. The variable holds a reference (pointer) to the memory location. Accessing these can be slower due to the additional level of indirection.

Value vs. Reference Semantics:

- Primitive Types: When assigned to another variable, a copy of the value is made. Changes to one variable do not affect the other.
- Reference Types: When assigned to another variable, both variables point to the same memory location. Changes made through one variable reflect in the other.

Mutability:

- Primitive Types: Immutable in most languages (though some languages may offer ways to "simulate" mutability).
- Reference Types: Mutable, allowing modifications to the data they point to without changing the reference.

Default Values:

- Primitive Types: Usually have default values (e.g., 0 for integers, false for booleans).
- Reference Types: Typically default to null or undefined if not explicitly initialized.

```
// Primitive example
public class PrimitiveExample {
    public static void main(String[] args) {
        int a = 5;
        int b = a; // b is now a copy of a
        b = 10; // changing b does not affect a

        System.out.println("Value of a: " + a); // Output: Value of a: 5
        System.out.println("Value of b: " + b); // Output: Value of b: 10
    }
}
```

```
// Reference example
public class ReferenceExample {
    public static void main(String[] args) {
        // Creating an object of MyClass
        MyClass obj1 = new MyClass();
        obj1.name = "Alice";

        // obj2 is a reference to the same object as obj1
        MyClass obj2 = obj1;
        obj2.name = "Bob"; // modifying obj2 affects obj1

        System.out.println("Value of obj1.name: " + obj1.name); // Output: Value of obj1.name: Bob
        System.out.println("Value of obj2.name: " + obj2.name); // Output: Value of obj2.name: Bob
    }
}

class MyClass {
    String name;
}
```

69. What are the Different Primitive Types and the memory they occupy?

In Java, primitive types are the most basic data types provided by the language. Each primitive type has a fixed size and a default value. Here is a list of Java's primitive types along with their sizes and default values:

1. byte

- Size: 8 bits (1 byte)
- Range: -128 to 127
- Default Value: 0

2. short

- Size: 16 bits (2 bytes)
- Range: -32,768 to 32,767
- Default Value: 0

3. int

- Size: 32 bits (4 bytes)
- Range: -2^31 to 2^31 - 1 (-2,147,483,648 to 2,147,483,647)
- Default Value: 0

4. long

- Size: 64 bits (8 bytes)
- Range: -2^63 to 2^63 - 1 (-9,223,372,036,854,775,808 to 9,223,372,036,854,775,807)
- Default Value: 0L

5. float

- Size: 32 bits (4 bytes)
- Range: Approximately ±3.40282347E+38F (6-7 significant decimal digits)
- Default Value: 0.0f

6. double

- Size: 64 bits (8 bytes)
- Range: Approximately ±1.79769313486231570E+308 (15 significant decimal digits)
- Default Value: 0.0d

7. char

- Size: 16 bits (2 bytes)
- Range: 0 to 65,535 (unsigned)
- Default Value: '\u0000' (null character)

8. boolean

- Size: The size is not precisely defined; it depends on the JVM. However, it typically takes up 1 bit of information, though it might be represented by 1 byte for simplicity in many implementations.
- Range: true or false
- Default Value: false

70. What is concept of Reflection in Java?

Reflection is a powerful feature in Java that allows you to inspect and manipulate the structure and behavior of classes, interfaces, fields, and methods at runtime. It's like a flashlight that lets you shine a light on the internal workings of your code, even if you don't know the names of those internal components beforehand.

How Reflection Works

• Introspection: Examining the structure of classes and objects. You can get information like class names, method signatures, field types, annotations, and more.

• Manipulation: Modifying the behavior of classes and objects at runtime. You can create new instances, invoke methods, access and modify fields, and even change access modifiers (within security constraints).

Common Use Cases of Reflection

1. Frameworks and Libraries:
   • Dependency Injection: Frameworks like Spring use reflection to instantiate and wire together components at runtime.
   • Testing Frameworks: JUnit and other testing tools use reflection to discover and run test methods.
   • ORM (Object-Relational Mapping): Libraries like Hibernate use reflection to map Java objects to database tables.
2. Tools and IDEs:
   • Debuggers: Use reflection to inspect objects and variables while your program is running.
   • IDEs: Use reflection to provide features like code completion, refactoring, and automatic generation of code.
3. Dynamic Loading and Configuration:
   • Plugins: You can load and configure plugins or modules at runtime without hardcoding their details.
   • Custom Annotations: Reflection helps process custom annotations to trigger specific behaviors.

```
import java.lang.reflect.Field;
import java.lang.reflect.Method;

class Company {
    private String name;

    public Company(String name) {
        this.name = name;
    }

    public String getName() {
        return name;
    }

    private void displayName() {
        System.out.println
                ("Private Method Invoked to display Company Name: " + name);
    }

}

public class ReflectionExample {

    // Using reflection to set a private field and invoke a private method
    public static void main(String[] args) throws Exception{
```

```
        // Create an instance of Company
        Company company = new Company("Youtube");
        System.out.println("Company Name: " + company.getName());

        // Change the name from Youtube to Ytube
        Field nameField = Company.class.getDeclaredField("name");
        nameField.setAccessible(true);
        nameField.set(company,"Ytube");
        System.out.println(company.getName());

        // Invoke the private method 'displayName'
        Method objMethod = Company.class.getDeclaredMethod("displayName");
        objMethod.setAccessible(true);
        objMethod.invoke(company);



    }
}
```

71. What Is the Difference Between an Inner Class and a Static Nested Class?

In Java, nested classes are classes defined within another class. There are two main types of nested classes: inner classes and static nested classes. Here's a detailed explanation of the differences between the two:

Inner Classes

Inner classes are non-static nested classes. They have the following characteristics:

- Associated with an Instance: Inner classes are tied to an instance of the outer class. This means they can access the outer class's instance variables and methods directly.

```
class OuterClass {
    class InnerClass {
        // Inner class code
    }
}
```

- Access to Outer Class Members: Inner classes have direct access to the outer class's instance members, including private ones.
- Instantiation: An inner class instance is created using an instance of the outer class.

```
OuterClass outer = new OuterClass();
OuterClass.InnerClass inner = outer.new InnerClass();
```

• Reference to Outer Class: An inner class has an implicit reference to its enclosing instance of the outer class.

Static Nested Classes

Static nested classes are static members of the outer class. They have the following characteristics:

• Not Associated with an Instance: Static nested classes do not have an implicit reference to an instance of the outer class. They can be instantiated without an instance of the outer class.

```
class OuterClass {
    static class StaticNestedClass {
        // Static nested class code
    }
}
```

• Access to Outer Class Members: Static nested classes can only access the static members of the outer class directly. To access instance members, they need an instance of the outer class.

• Instantiation: A static nested class instance is created without needing an instance of the outer class.

```
OuterClass.StaticNestedClass nested = new OuterClass.StaticNestedClass();
```

• No Reference to Outer Class: Static nested classes do not have an implicit reference to an outer class instance.

```
// Inner Class Example
public class OuterClass {
    private int outerField = 10;

    class InnerClass {
        void display() {
            System.out.println("Outer field: " + outerField);
        }
    }

    public static void main(String[] args) {
        OuterClass outer = new OuterClass();
        OuterClass.InnerClass inner = outer.new InnerClass();
        inner.display(); // Output: Outer field: 10
    }
}
```

```
// Static Nested Class Example:
public class OuterClass {
    private static int staticOuterField = 20;

    static class StaticNestedClass {
        void display() {
```

```
            System.out.println("Static outer field: " + staticOuterField);
        }
    }

    public static void main(String[] args) {
        OuterClass.StaticNestedClass nested = new OuterClass.StaticNestedClass();
        nested.display(); // Output: Static outer field: 20
    }
}
```

72. What is difference between equals() and ==?

In Java, equals() and == are used for comparison, but they serve different purposes and behave differently. Here's a detailed explanation of the differences between equals() and ==:

== Operator

Primitive Types: When used with primitive types, == compares the actual values.

```
int a = 5;
int b = 5;
System.out.println(a == b); // Output: true
```

Reference Types: When used with reference types (objects), == compares the references (memory addresses) to check if they point to the same object.

```
String s1 = new String("hello");
String s2 = new String("hello");
System.out.println(s1 == s2); // Output: false (different objects in memory)
```

equals() Method

1. Object Class: The equals() method is defined in the Object class. By default, it behaves like ==, comparing object references.

```
class MyClass {
    int value;
}

MyClass obj1 = new MyClass();
MyClass obj2 = new MyClass();
System.out.println(obj1.equals(obj2)); // Output: false (default behavior)
```

2. Overridden Method: Many classes, like String, Integer, and custom classes, override equals() to compare the actual content (state) of the objects.

```
String s1 = new String("hello");
String s2 = new String("hello");
System.out.println(s1.equals(s2)); // Output: true (content is the same)
```

3. Custom Classes: When you create your own classes, you can override equals() to define what it means for two instances of your class to be "equal".

```
class MyClass {
    int value;

    MyClass(int value) {
        this.value = value;
    }

    @Override
    public boolean equals(Object obj) {
        if (this == obj) return true;
        if (obj == null || getClass() != obj.getClass()) return false;
        MyClass myClass = (MyClass) obj;
        return value == myClass.value;
    }
}

MyClass obj1 = new MyClass(10);
MyClass obj2 = new MyClass(10);
System.out.println(obj1.equals(obj2)); // Output: true (values are the same)
```

Summary of Differences

1. Comparison Level:
   - ==: Compares references (memory addresses) for objects; compares actual values for primitives.
   - equals(): Compares the content/state of objects (if overridden); compares references if not overridden.
2. Default Behavior:
   - ==: Always checks for reference equality for objects.
   - equals(): By default (in Object class), checks for reference equality, but can be overridden for content equality.
3. Usage:
   - Use == for: Comparing primitive values. Checking if two references point to the same object.
   - Use equals() for: Comparing the content/state of two objects. Implementing logical equality in custom classes.

73. What Is an Anonymous Class and what is it's Use Case?

An anonymous class in Java is a type of inner class without a name. It is used to instantiate a class defined in

the method body, usually for implementing an interface or extending a class with minor modifications. Anonymous classes are often used for short-lived implementations that are not reused elsewhere.

Characteristics of Anonymous Classes

- No Name: As the name suggests, anonymous classes do not have a name. They are defined and instantiated in a single expression.
- Syntax: Typically created using new followed by the interface or class they extend or implement.

```
new InterfaceOrClass() {}
```

- Instantiation: They are usually instantiated where they are defined, often in the context of a method call or a variable initialization.
- Single Use: Anonymous classes are intended for use in a single context and are not reusable.

Use Cases for Anonymous Classes

1. Event Handling: Commonly used in GUI applications for handling events (e.g., button clicks).

```
import javax.swing.JButton;
import javax.swing.JFrame;

public class AnonymousClassExample {
    public static void main(String[] args) {
        JFrame frame = new JFrame("Anonymous Class Example");
        JButton button = new JButton("Click Me");

        button.addActionListener(new ActionListener() {
            @Override
            public void actionPerformed(ActionEvent e) {
                System.out.println("Button was clicked!");
            }
        });

        frame.add(button);
        frame.setSize(300, 200);
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        frame.setVisible(true);
    }
}
```

2. Thread Creation: Simplifies thread creation without defining a separate class.

```
public class AnonymousClassExample {
    public static void main(String[] args) {
        new Thread(new Runnable() {
            @Override
            public void run() {
                System.out.println("Thread is running");
```

```
            }
        }).start();
    }
}
```

3. Customizing Behavior: Allows for quick customization of existing classes or interfaces.

```
import java.util.ArrayList;
import java.util.List;

public class AnonymousClassExample {
    public static void main(String[] args) {
        List<String> list = new ArrayList<String>() {
            @Override
            public boolean add(String s) {
                System.out.println("Adding element: " + s);
                return super.add(s);
            }
        };

        list.add("Hello");
        list.add("World");
    }
}
```

74. What Is a Marker Interface and examples of Marker Interfaces in Java?

Marker Interface in Java

A marker interface in Java is an interface with no methods or constants. It serves as a means to signal to the JVM or other code that a class implementing this interface has a particular property or should be treated in a special way. Marker interfaces are used to provide metadata to objects.

Purpose of Marker Interfaces

- Type Identification: Identifying the type of an object at runtime.
- Special Behavior: Indicating that a class should be processed in a special manner by certain Java APIs or libraries.
- Security and Serialization: Controlling the serialization mechanism or ensuring certain security measures.

Examples of Marker Interfaces in Java

1. Serializable: This marker interface is used to indicate that a class can be serialized, i.e., its state can be converted into a byte stream and subsequently restored.

```
import java.io.Serializable;
```

```
public class MyClass implements Serializable {
    private static final long serialVersionUID = 1L;
    private int id;
    private String name;

    // Constructor, getters, and setters
}
```

2. Cloneable: This marker interface is used to indicate that a class allows a clone of its objects to be created. Implementing this interface signals that the Object.clone() method can be used on instances of the class.

```
public class MyClass implements Cloneable {
    private int id;
    private String name;

    // Constructor, getters, and setters

    @Override
    protected Object clone() throws CloneNotSupportedException {
        return super.clone();
    }
}
```

3. Remote: This marker interface is used to indicate that an interface represents a remote object that can be invoked from a non-local virtual machine. It is a part of Java RMI (Remote Method Invocation).

```
import java.rmi.Remote;
import java.rmi.RemoteException;

public interface MyRemoteInterface extends Remote {
    void myRemoteMethod() throws RemoteException;
}
```

Implementation and Use Case

```
// Serializable Example:
import java.io.*;

public class SerializableExample {
    public static void main(String[] args) {
        MyClass obj = new MyClass(1, "John");

        // Serialize the object
        try (ObjectOutputStream out = new ObjectOutputStream(new FileOutputStream("object.ser"))) {
            out.writeObject(obj);
        } catch (IOException e) {
            e.printStackTrace();
```

```
        }

        // Deserialize the object
        try (ObjectInputStream in = new ObjectInputStream(new FileInputStream("object.ser"))) {
            MyClass deserializedObj = (MyClass) in.readObject();
            System.out.println("Deserialized object: " + deserializedObj.getName()); // Deserialized ob
        } catch (IOException | ClassNotFoundException e) {
            e.printStackTrace();
        }
    }
}

class MyClass implements Serializable {
    private static final long serialVersionUID = 1L;
    private int id;
    private String name;

    public MyClass(int id, String name) {
        this.id = id;
        this.name = name;
    }

    public int getId() {
        return id;
    }
public String getName() {
        return name;
    }
}
```

```
// Cloneable Example:
public class CloneableExample {
    public static void main(String[] args) {
        MyClass obj1 = new MyClass(1, "John");
        try {
            MyClass obj2 = (MyClass) obj1.clone();
            System.out.println("Cloned object: " + obj2.getName()); // Cloned object: John
        } catch (CloneNotSupportedException e) {
            e.printStackTrace();
        }
    }
}

class MyClass implements Cloneable {
    private int id;
    private String name;

    public MyClass(int id, String name) {
        this.id = id;
```

```
        this.name = name;
    }

    public int getId() {
        return id;
    }

    public String getName() {
        return name;
    }

    @Override
    protected Object clone() throws CloneNotSupportedException {
        return super.clone();
    }
}
```

75. What Is an Immutable Class and how to create it?

An immutable class in Java is a class whose instances cannot be modified after they are created. Once an object is created, its state (fields) cannot be changed. Immutable objects are inherently thread-safe and can be shared freely among multiple threads without synchronization concerns.

Characteristics of Immutable Classes

- Final Class: The class should be declared as final so that it cannot be subclassed.
- Private Final Fields: All fields should be declared as private and final to ensure they are not modified after initialization.
- No Setter Methods: The class should not provide any methods that modify fields or objects referred to by fields.

Example of an Immutable Class

```
import java.util.Date;

public final class ImmutablePerson {
    private final String name;
    private final int age;
    private final Date birthDate; // mutable object

    public ImmutablePerson(String name, int age, Date birthDate) {
        this.name = name;
        this.age = age;
        // Creating a defensive copy of the mutable Date object
        this.birthDate = new Date(birthDate.getTime());
    }
```

```
    public String getName() {
        return name;
    }

    public int getAge() {
        return age;
    }

    public Date getBirthDate() {
        // Returning a defensive copy of the mutable Date object
        return new Date(birthDate.getTime());
    }
}
```

Explanation of the Example

- Class Declaration: public final class ImmutablePerson: The class is declared as final.
- Private Final Fields: private final String name; private final int age; private final Date birthDate;
- Constructor: The constructor initializes all fields. For the mutable Date object, a defensive copy is made using new Date(birthDate.getTime()).
- Getter Methods: getName(), getAge(), and getBirthDate() methods provide access to the fields. For the Date field, getBirthDate() returns a new Date object to prevent modification of the internal birthDate field.

76. Is it possible to Override a Static Method?

In Java, it is not possible to override a static method. Static methods belong to the class rather than any instance of the class. Overriding is a concept related to instance methods, where a subclass provides a specific implementation for a method that is already defined in its superclass.

Static Method Behavior

- Class-Level Methods: Static methods are class-level methods, meaning they are associated with the class itself rather than any particular object instance.
- Hiding, Not Overriding: When a static method is defined in a superclass and a subclass defines a static method with the same name and parameter list, it is known as method hiding, not method overriding. The static method in the subclass hides the static method in the superclass.
- Compile-Time Binding: Static methods are bound at compile-time, whereas instance methods are bound at runtime. This is why you can't override static methods, but you can hide them.

Example of Static Method Hiding

```
class Parent {
    public static void staticMethod() {
        System.out.println("Static method in Parent class");
    }
```

```
    public void instanceMethod() {
        System.out.println("Instance method in Parent class");
    }
}

class Child extends Parent {
    // This hides the static method in Parent, it does not override it
    public static void staticMethod() {
        System.out.println("Static method in Child class");
    }

    // This overrides the instance method in Parent
    @Override
    public void instanceMethod() {
        System.out.println("Instance method in Child class");
    }
}

public class Test {
    public static void main(String[] args) {
        Parent parent = new Parent();
        Parent childAsParent = new Child();
        Child child = new Child();

        parent.staticMethod(); // Output: Static method in Parent class
        childAsParent.staticMethod(); // Output: Static method in Parent class
        child.staticMethod(); // Output: Static method in Child class

        parent.instanceMethod(); // Output: Instance method in Parent class
        childAsParent.instanceMethod(); // Output: Instance method in Child class
        child.instanceMethod(); // Output: Instance method in Child class
    }
}
```

Explanation

1. Static Method Calls:
   - Parent.staticMethod() prints "Static method in Parent class".
   - childAsParent.staticMethod() (where childAsParent is of type Parent) also prints "Static method in Parent class" because static methods are resolved at compile time based on the reference type, not the actual object.
   - child.staticMethod() (where child is of type Child) prints "Static method in Child class".
2. Instance Method Calls:
   - parent.instanceMethod() prints "Instance method in Parent class".
   - childAsParent.instanceMethod() prints "Instance method in Child class" because instance methods are resolved at runtime based on the actual object.

• child.instanceMethod() prints "Instance method in Child class".

77. How to Compare Two Enum Values?

In Java, enums are a special type of class used to define collections of constants. Comparing two enum values is straightforward and can be done using either the == operator or the equals() method. Both approaches are valid, but there are subtle differences in their usage and implications.

Comparing Enum Values Using ==

The == operator is the recommended way to compare enum values. It checks for reference equality, which is appropriate for enums because each enum constant is a singleton (there is only one instance of each enum constant).

```
public enum Color {
    RED, GREEN, BLUE;
}

public class EnumComparison {
    public static void main(String[] args) {
        Color color1 = Color.RED;
        Color color2 = Color.RED;
        Color color3 = Color.GREEN;

        if (color1 == color2) {
            System.out.println("color1 and color2 are the same");
        } else {
            System.out.println("color1 and color2 are different");
        }

        if (color1 == color3) {
            System.out.println("color1 and color3 are the same");
        } else {
            System.out.println("color1 and color3 are different");
        }
    }
}
// Output
// color1 and color2 are the same
// color1 and color3 are different
```

Comparing Enum Values Using equals()

The equals() method can also be used to compare enum values. It checks for logical equality, which in the case of enums is the same as reference equality. However, using equals() is less common because == is more concise and clearer for enums.

```
public class EnumComparison {
    public static void main(String[] args) {
```

```
        Color color1 = Color.RED;
        Color color2 = Color.RED;
        Color color3 = Color.GREEN;

        if (color1.equals(color2)) {
            System.out.println("color1 and color2 are the same");
        } else {
            System.out.println("color1 and color2 are different");
        }

        if (color1.equals(color3)) {
            System.out.println("color1 and color3 are the same");
        } else {
            System.out.println("color1 and color3 are different");
        }
    }
}

// Output
// color1 and color2 are the same
// color1 and color3 are different
```

Summary

- Using ==: This is the preferred way to compare enum values because it checks for reference equality and is more concise.
- Using equals(): This also works because it checks for logical equality, which is the same as reference equality for enums. However, it is less commonly used.

78. What is the purpose of the this keyword?

In Java, the this keyword is a reference to the current object within an instance method or constructor. It has several purposes, all related to distinguishing between instance variables and parameters or to explicitly referring to the current object.

Purpose of the this Keyword

1. To Refer to Instance Variables: When a local variable or parameter has the same name as an instance variable, this can be used to refer to the instance variable.

```
public class MyClass {
    private int value;

    public void setValue(int value) {
        this.value = value; // "this.value" refers to the instance variable
    }
}
```

2. To Call Another Constructor: Within a constructor, this() can be used to call another constructor in the same class.

```
public class MyClass {
    private int value;

    public MyClass() {
        this(0); // Calls the constructor with an int parameter
    }

    public MyClass(int value) {
        this.value = value;
    }
}
```

3. To Pass the Current Object as a Parameter: It can be used to pass the current object as a parameter to another method or constructor.

```
public class MyClass {
    public void doSomething() {
        AnotherClass.process(this); // Passes the current object to the process method
    }
}
```

4. To Return the Current Object: It can be used to return the current object from a method, enabling method chaining.

```
public class MyClass {
    public MyClass increment() {
        this.value++;
        return this; // Returns the current object
    }
}
```

Summary

The this keyword in Java is a reference to the current object and is primarily used to disambiguate between instance variables and parameters, or to explicitly refer to the current object. It's a fundamental tool for managing object-oriented programming concepts such as encapsulation, constructor chaining, and method chaining.

79. How does the final keyword affect class inheritance and method overriding?

In Java, the final keyword has different effects depending on whether it is applied to a class, method, or variable.

Final Class

- When a class is declared as final, it cannot be subclassed.
- Any attempt to extend a final class will result in a compilation error.

```
final class FinalClass {
    // Class implementation
}

// Error: Cannot inherit from final FinalClass
class Subclass extends FinalClass {
    // Subclass implementation
}
```

Final Method

- When a method is declared as final in a superclass, it cannot be overridden in subclasses.
- Subclasses are not allowed to provide a different implementation of a final method.

```
class Parent {
    public final void finalMethod() {
        // Method implementation
    }
}

class Child extends Parent {
    // Error: Cannot override final method finalMethod() in Parent
    public void finalMethod() {
        // Overridden method implementation
    }
}
```

Effects on Inheritance and Method Overriding

- Prevents Inheritance: Declaring a class as final prevents other classes from inheriting from it, ensuring that the behavior of the class cannot be modified or extended.
- Prevents Method Overriding: Declaring a method as final prevents subclasses from overriding it. This is useful when a method in a superclass is considered complete and should not be modified in subclasses.

When to Use final

- Use final for classes or methods when you want to prevent any modifications or extensions to them.
- It's often used for utility classes, immutable classes, or critical methods that should not be overridden.

80. What Are the Wrapper Classes and what is autoboxing and unboxing in Java?

In Java, the primitive data types (e.g., int, double, boolean) are not objects, which means they cannot be used in scenarios where objects are required, such as collections like ArrayList or methods that expect objects. Wrapper classes are a set of classes provided by Java to represent each of the primitive data types as objects. These classes provide utility methods and enable primitive types to be used in contexts where objects are required.

Wrapper Classes in Java

- Integer: Represents the int primitive type.
- Double: Represents the double primitive type.
- Boolean: Represents the boolean primitive type.
- Character: Represents the char primitive type.
- Byte: Represents the byte primitive type.
- Short: Represents the short primitive type.
- Long: Represents the long primitive type.
- Float: Represents the float primitive type.

Autoboxing and Unboxing

1. Autoboxing: Autoboxing is the automatic conversion of primitive data types into their corresponding wrapper class objects. It happens when the primitive data types are used in contexts where objects are required, such as adding them to collections or passing them to methods that expect objects.

```
int primitiveInt = 10;
Integer wrapperInt = primitiveInt; // Autoboxing
```

2. Unboxing: Unboxing is the automatic conversion of wrapper class objects back to their corresponding primitive data types. It happens when wrapper class objects are used in contexts where primitive data types are expected, such as performing arithmetic operations.

```
Integer wrapperInt = 20;
int primitiveInt = wrapperInt; // Unboxing
```

```
import java.util.ArrayList;

public class AutoboxingUnboxingExample {
    public static void main(String[] args) {
        // Autoboxing: converting int to Integer
        int primitiveInt = 42;
        Integer wrapperInt = primitiveInt;

        // Unboxing: converting Integer to int
        Integer wrapperInt2 = 50;
        int primitiveInt2 = wrapperInt2;

        // Autoboxing in collections
```

```
        ArrayList<Integer> list = new ArrayList<>();
        list.add(1); // Autoboxing
        list.add(2); // Autoboxing

        // Unboxing from collections
        int first = list.get(0); // Unboxing
        int second = list.get(1); // Unboxing

        System.out.println("Autoboxing: " + wrapperInt); // Autoboxing: 42
        System.out.println("Unboxing: " + primitiveInt2); // Unboxing: 50
        System.out.println("Autoboxing in collection: " + list); // Autoboxing in collection: [1, 2]
        System.out.println("Unboxing from collection: " + first + ", " + second); // Unboxing from coll
    }
}
```

In this example, autoboxing occurs when assigning the primitive int to the Integer object and when adding integers to the ArrayList. Unboxing occurs when assigning the Integer object to a primitive int and when retrieving integers from the ArrayList. Autoboxing and unboxing simplify the coding process by allowing primitive data types and their corresponding wrapper classes to be used interchangeably.

81. What Is a Var-Arg and how to use it? What are benefits and limitations?

A varargs (variable-length arguments) parameter allows a method to accept zero or more arguments of a specified type. It's declared by specifying three dots (...) after the parameter type in the method signature. Varargs provide flexibility in the number of arguments that can be passed to a method.

```
public void methodName(Type... parameterName) {
    // Method implementation
}
```

Usage: You can pass any number of arguments of the specified type (or none at all). Inside the method, the varargs parameter behaves like an array of the specified type.

```
public class VarargsExample {
    public static void main(String[] args) {
        printNumbers(1, 2, 3, 4, 5);
        printNumbers(); // Calling with no arguments
    }

    public static void printNumbers(int... numbers) {
        System.out.println("Printing numbers:");
        for (int num : numbers) {
            System.out.println(num);
        }
    }

  // Output
```

```
// Printing numbers:
// 1
// 2
// 3
// 4
// 5
// Printing numbers:
}
```

Benefits of Varargs

• Flexibility: Varargs allow methods to accept a variable number of arguments, providing flexibility in method calls.

• Simplified Syntax: Varargs simplify the syntax for methods that accept multiple arguments, avoiding the need for arrays or overloaded methods.

• Reduced Boilerplate Code: Varargs eliminate the need to create and manipulate arrays manually when passing multiple arguments.

Limitations of Varargs

• Positional Arguments Only: Varargs must be the last parameter in a method signature, and you cannot have more than one varargs parameter in a method.

• Performance Overhead: Varargs are implemented using arrays internally, which may result in a slight performance overhead compared to fixed-size parameter lists.

• Type Safety: Varargs can accept arguments of any length, potentially leading to type-related errors if not used carefully.